Engineering Thesis Project

HAND-GESTURE DETECTION AND ASL TRANSLATION APPLICATION: INTEGRATION OF IMAGE PROCESSING AND SENSORS

Academic Supervisor: Dr. Pham Xuan Tung

Student Name: Tran Anh Phi

Student ID: BI12-343

Sunday, 6 October 2024

Declaration of Originality

I hereby declare that the research carried out in this thesis was in accordance with the regulations of the University of Science and Technology of Hanoi. The work presented here is my own and has been generated as the result of original research done by myself. All external sources used in the process have been fairly and unambiguously cited. Furthermore, this research has not been submitted to any other institution to obtain another degree or qualification. This is a copy of the thesis, with final revisions included.

Signature:

Tran Anh Phi

Sunday, 6 October 2024

Acknowledgments

This study was made possible through the aid of incredibly brilliant engineers and researchers who dedicated their efforts to educating others and making applied engineering and computer science more accessible for people like myself. I would like to express my sincere gratitude to all the professionals listed in the references for their incredibly informative and instructional resources that helped to assist in the completion of this research project. In addition, I would like to acknowledge the support from my instructor, Dr. Pham Xuan Tung, and the University of Science and Technology of Hanoi for providing the necessary facilities needed to carry out this study. I would like to extend my sincere gratitude to my friends and family for their endless support for this project. Lastly, my appreciation goes towards everyone who directly or indirectly supported me throughout my research. Thank you all for your tremendous contribution to this project.

Abstract

The language barrier presents a significant challenge to effective communication and relationship-building, particularly for individuals who are unable to communicate verbally, such as those with hearing impairments or language disorders who rely on sign language. Acquiring a new language demands time and resources, which may not be readily available in our fast-paced society. Various initiatives have been undertaken to bridge the communication gap between sign language users and non-users, including the use of data gloves, sensors, and machine learning for image processing. This study proposes a new hybrid approach that combines computational image analysis and sensor-focused techniques to facilitate a better understanding of sign language for non-users. The objective is to enhance detection ranges, improve performance in more complex environments, and promote accessibility for individuals who primarily use sign language for communication.

Contents

Declaration of Originality ii
Acknowledgments iii
Abstract iv
Chapter 1: Introduction vii
1. Background of the study viii
2. Statement of the problem viii
3. Objective ix
4. Limitations and Scopes ix
Chapter 2: Literature Review x
1. Sign Language Recognition Overview x
1.1. Non-vision-based approach xi
1.2. Vision-based approach xii
2. Application Areas xiv
Chapter 3: Methodology xv
1. Proposed System xv
2. Sign language system: American Sign Language system (ASL) xvi
3. Mediapipe xvi
4. Sign recognition system description xvii
4.1. Components xvii
4.2. System architecture xxiii
Chapter 4: Results and Discussions xxxviii
1. Overview xxxviii
2. Performance Analysis xxxviii
3. Limitations xl
Chapter 5: Conclusion xlii
References xliii

List of Figures

Figure 1: 21 hand landmarks localized by MediaPipe hand detection model (Source: [22]) xiii
Figure 2: Block diagram of the processes and architecture of the system xv
Figure 3: The American Sign Language alphabet (Source: Stock Images) xvi
Figure 4: The mechanical specifications of the Raspberry Pi 4 (Source: [27]) xviii
Figure 5: Raspberry Pi 4's essential components (Source: Raspberry Pi official website) xviii
Figure 6: Mechanical specifications of the Picamera module (Source: [29]) xix
Figure 7: The components and pins of the Arduino Nano (Source: [30]) xix
Figure 8: The measurements of the Arduino Nano (Source: [30]) xx
Figure 9: The components and pins of the photoresistor module (Source: [33]) xxi
Figure 10: Electrical diagram of the photoresistor module, D1 is Power LED, D2 is Status LED (Source: [33]) xxii
Figure 11: (left): The inner components of a microservo, (right): The pan and tilt unit and their rotational axes (Source: Internet) xxii
Figure 12: The architecture of the neural network xxiii
Figure 13: processes involved in developing the hand sign recognition model for Raspberry Pi xxiii
Figure 14: Mediapipe's process for hand detection and recognition xxiv
Figure 15: Mediapipe's graph for hand tracking, consisting of two subgraphs one for hand detection and another for landmarks computation, the landmark model output dictates when the hand detection model is activated. (Source: [22]) xxiv
Figure 16: Video capture of hand recognition during data collection of key point coordinates xxvi
Figure 17: The model summary of the neural network xxviii
Figure 18: The graph for the ReLU Activation Function (Source: [40]) xxix
Figure 19: The graph for the Softmax Activation Function (Source: [40]) xxix
Figure 20: Input and Output of the Softmax Activation Function (Source: [41]) xxx
Figure 21: The formula for calculating new weights in an Optimizer (Source: [43]) xxx
Figure 22: The layers and parameters of a Feedforward Neural Network xxxi
Figure 23: The logic algorithm of a Loss Function (Source: [44]) xxxii
Figure 24: A neural network with and without the application of dropout layers (Source: [48]) xxxiv
Figure 25: A schematic of how the components are wired together xxxvi
Figure 26: Hardware schematics, simplified xxxvii
Figure 27: The design of the prototype translator device xxxvii
Figure 28: Recognition of the first 6 letters in the ASL alphabet xxxviii
Figure 29: (Left): The newly proposed signing option (Right): The original posture xxxix
Figure 30: (Left): The accuracy graph after training and testing (Right): The loss of the model xxxix
Figure 31: The confusion matrix and the classification report of the model after 400 epochs xl

List of *Tables*

Table 1: First column: order of sign, Second and subsequent columns: landmark coordinates (normalized) ..... *xxvi*
Table 2: The normalization process of landmark coordinates ..... *xxvii*

List of Equations

Equation 1 xxviii
Equation 2 xxix
Equation 3 xxxiii
Equation 4 xxxiii

Chapter 1: Introduction

1. Background of the study

Out of many existing restrictions preventing effective communication and the formation of interpersonal relationships between individuals, the language barrier creates one of society's most serious limitations. Specifically for the ones without the ability to articulate verbally, this challenge is proven to be even more relevant. People with hearing impairment or language disorders employ sign language to convey their thoughts and emotions. For the average person, learning and understanding a new language requires time and effort that may not be readily available in our increasingly busy society. The field of assistive technology has been advancing at an impressive rate in recent decades, with the development of various tools, devices, and services to aid individuals with disabilities.

According to the World Health Organization (WHO), hearing impairment impacts over 430 million people worldwide.[1]. In stark contrast, the report also demonstrated that supportive products for people with ineffective hearing loss are distributed and benefited in middle to low-income countries by only 3% (2019). Much research has been conducted in an attempt to bridge the gap between sign language users and non-users, the most notable solutions are between the utilization of data gloves and sensors or the ever-evolving subfield of machine learning that is image processing. Although there are proven benefits to both of these methods, this study aims to examine a new approach, a hybrid between the heavy computational image analysis and the sensor-focused idea of the data gloves.

2. Statement of the problem

Effective communication is an essential part of human interaction, and the absence of one means to perceive the world facilitates problems for less fortunate individuals. Consequently, the development of low-cost and effective solutions to improve and simplify special communication becomes a necessity to improve the quality of life of people with disabilities and those with limited financial means. Many studies have explored the possibilities in sign language translation, commonly utilizing data gloves or machine learning in various applications. Prosthetics and wearable devices are used in conjunction with flexible sensors that measure the bending degrees of our fingers and accelerometers to measure different hand positions and return readings based on the data collected. Alternatively, image processing is an ongoing and rapidly developed field, producing many promising results with high-accuracy predictions and real-time translation of hand signs, using Hand Recognition (HR) models built upon Convolutional Neural Networks (CNN) and frameworks such as TensorFlow. Although there are proven benefits to both of these methods, neither is hardly perfect, introducing various drawbacks depending on the approach. Although efficient enough in recognizing basic gestures and signs, the data glove lacks the ability to portray the intricacies of more complicated hand postures, restrictiveness can be a factor [2] coupled with the sensitive sensors integrated into the design, which can be subjected to outside interference. For all its disadvantages, the data glove method carries certain simplicities that are not necessarily present in the Deep Learning and Computer Vision approach. The advantages of using image processing for these types of tasks are evidenced in its intuitiveness and high accuracy even when

complex gestures are brought into the equation. The two major problems this model faces are the restrictive space for movement within the provided frame of the device capturing the image and changing light conditions or non-uniform backgrounds leading to inaccuracies in the recognition process [3], [4].

The focus here is to embrace the most effective parts of each method while using the one to improve where the other ineffectiveness comes in. Moreover, research on a lower-cost device that is capable of translating sign language in real-time while employing a hybrid between the approach of computer vision and the sensor-based idea is still limited.

3. Objective

Despite the growing need for assistive technology, there is a lack of research on developing economical and efficient assistive methods for hearing-impaired individuals. This project's objective is to develop a low-cost and relatively effective solution that can aid communication for the hearing-impaired and create an accessible gateway for the general public to familiarize themselves with sign languages. Utilizing sensors and servo motors with microcontrollers to reinforce the effectiveness of a machine algorithm that can translate sign language into English via real-time camera inputs, disabling many weaknesses presented by these applications when operated individually, enabling them to work together in support of those with disabilities to communicate more effectively with voice-activated technologies and individuals who are unfamiliar with their respective sign language systems, as well as exploring a new and possibly more effective way for people to be introduced to and learn signs, bridging the gap between communities, enhance inclusivity and accessibility for individuals whose primary means of communication involves sign languages.

4. Limitations and Scopes

With considerations for the feasibility and time duration given to this project, the expected outcome is to successfully develop a relatively affordable and user-friendly sign language translation prototype with an initial focus on the ability to recognize the 26 letters in the American Sign Language (ASL) alphabet. The objective is to achieve 80% accuracy in gesture recognition under various lighting conditions, with a translation response time of less than one second. Furthermore, comfortable, aesthetically pleasing, and easy-to-use designs will be prioritized. This project reaches completion in 5 stages, these are:

I. Data Acquisition: Collect a comprehensive American Sign Language (ASL) sign dataset with corresponding text/speech labels.
II. Preprocessing and Feature Extraction: Design and develop circuitry for sensor implementation and create image processing algorithms to isolate and analyze hand signs.
III. Machine Learning Model training: Train a classifier using the extracted features and corresponding ASL translations.
IV. System Integration: Combine the components into a real-time ASL translation application.
V. Testing and Refinement: Evaluate the system's accuracy and continuously improve performance.

Chapter 2: Literature Review

1. Sign Language Recognition Overview

Research on Sign Language Recognition applications has been rising in popularity since the inception of numerous innovative tools and resources in fields of assistive technology development. Although countless research papers have been published on the subject, the practicality of a Sign Language Translator is still questionable due to many inherent complexities that come with certain methodologies applied to developing these devices[5]. Nevertheless, it is not beyond the realms of possibility to conduct an implementable design of an automated Sign Language Translator, specifically focusing on American Sign Language (ASL) in this instance. Many models have been proposed throughout the course of years of research with respectable results. The two main approaches that define this field of study are Computer vision-based gesture recognition and non-vision-based approaches i.e using Data Gloves[6]. The most typical example of a sensor-based Hand Recognition device is the aforementioned Data Gloves method, employing multiple-axis sensors attached to a wearable gadget, detecting any bending of the fingers or the angles of the joints. Additionally, spatial tracking equipment like accelerometers or gyroscope sensors are also considered for the measurement of the positions of the hand concerning the space surrounding it. Certain types of gloves offer Virtual Reality applications, gestures are recorded and transmitted to the computer through the glove in real-time, allowing human-computer interaction with virtual feedback. Although numerous new algorithms and designs are continuously being proposed with varying degrees of practicality and robustness in performance, the disadvantages of this method when considering the user experience are still less than favorable when compared to its more intuitive alternative, specifically applications based on Deep Learning algorithms and Computer Vision.

Object detection and recognition has been a rapidly developing field of study in recent years with the constant maturing of Computer Vision technology and other Deep Learning applications, resulting in countless tools being developed for the purpose of identifying anything that can be captured with a camera, whether it be people, animals or inanimate objects. This development ignites an array of ideas for applications that can be used for education and translation services, one of which is the research into Hand Recognition (HR). Prior to the integration of Deep Learning in image processing, there were methods by which one could program a machine to determine the shapes of hands in a captured digital frame. However, these were highly sophisticated heuristics that demanded to be performed in controlled laboratory environments with the use of depth sensors[7]. The inception of Deep Learning and Neural Networks brought about a new approach to Computer Vision by replacing depth sensors with RGB cameras, which is a more affordable and abundant alternative. This opens the door for an increase in the research interest in the digital interpretation of real-time hand movements, actively increasing the accuracy of such models, with a recognition rate ranging from 69% to 98%, averaging 88.8% for signer dependent as reported in a 2020 study[8]. Given the more restrictive nature of devices that require physical contact and connections, vision-based options for hand detection and recognition allow for more comfort and convenience for the benefactor [6]. In the following sections, both approaches to hand tracking and recognition and their respective ongoing developments will be explored further in detail.

1.1. Non-vision-based approach

Attachable sensor-based hand gesture classification devices are one of the earliest ideas at the genesis of all hand detection research, predating the conception and domination of Machine Learning and Computer Vision developments in later decades. One of the advantages offered by non-vision-based hand detection is the imperviousness to complex environmental interruptions such as the color of the background or the overall brightness in the atmosphere of the subject's hand. Area of coverage is also an important aspect to consider when adopting detection models. Due to being attached to the user, sensor-based devices offer a larger range of operations than their vision-based counterpart. At their most basic, Data gloves consist of the most essential motion sensors like gyroscopes or accelerometers, flex sensors are also largely employed for the task of measuring the bending of fingers, although numerous other designs that replace the flex sensor for other alternatives have been proposed with varying degrees of relative success. When considering the glove-based approach to hand gesture recognition, most concept proposals can be fundamentally divided into two major categories of research, designs that heavily rely on the employment of flex sensors and those that opt to adopt a less conventional approach. While maintaining the design of a wearable glove, some groups have proposed using several different kinds of instruments including capacitive touch sensors[9], piezo-resistive polymer sensors[10], or even LED-LDR pairs in order to conduct a binary system on each finger where 0 and 1 correspond to a state of bent or neutral, respectively[11]. Although these innovations all held great potential, none became the new standard when it came to sensor-based hand recognition techniques and developed further past its initial conceptualization.

The flex sensor-incorporated model is arguably the most popular and widely adopted way of designing a sign language recognition glove, with many papers on projects using similar designs still being published yearly with minor differences in between, with most relevant articles dating as early as 2014 [12] while the latest dates to 2024 at the time of writing this thesis. Flex sensors are helpful in performing tasks involving the measurement and categorization of certain bending degrees, and determining the posture according to the resistance values of the sensor. Multiple notable projects [13], [14], [15], [16] with respectable results have implemented this method. In [14], utilizing an Arduino microcontroller, five flex sensors were arranged on top of each finger to receive information about the differences in joint movements in addition to a better classification model using traditional machine learning for better results. Alternatively, we have another similar but slightly modified proposal with [15] and [16] where the main differentiation is the wireless communication ability between the glove and smartphones or the inclusion of an embedded LCD. One of the most recent glove-based studies with noteworthy developments employed a different and enhanced kind of bending sensor with sawtooth structures, one fabricated from liquid metal with great tensile strength and improved conductivity [13]. An increase in detection sensitivity and variation in resistance values was reported for this model. Overall, this approach to using physically bendable sensors offers simplicity in its hardware, and fabrication and has the potential for improvements in different applications. However, when aspects such as different hand geometries of the benefactors and the more complicated motions and gestures are brought into the equation, it is inarguably more difficult to maintain a consistent efficiency between users, and this is without considering the practicality in the comfortability, with reports about device putting strain on the user and inconvenience in performing signs while wearing glove-based products.

1.2. Vision-based approach

Although device-based systems initiated the groundwork for computerized hand gesture classification, since their conception, the more recent advancements made in the fields of Deep Learning and Neural Networks gave way to the development of computer programs that can accurately and efficiently detect as well as recognize more complex hand positions than any wearable options could have offered. Additionally, when considering the practicality of glove-based products in real-life circumstances, a vision-based approach remains preferable due to their intuitiveness and unbothered by having the user's movement be restricted by heavy or bulky attachments, lending more potential for outdoor applications[8]. Given these reasons, it is not surprising that one may find the majority of studies done on hand gesture recognition were based on Computer Vision and Deep Learning rather than the sensor-based approach of the past. Human-computer interactive technology has seen serious signs of progress in the last decade due to the ever-enhancing processing power of modern computers, allowing for better data collection and utilization. Generally, any design for a gesture recognition system based on visual input usually contains the following procedures: (1) *data acquisition and pre-processing*; (2) *feature extraction*, and (3) *classification or recognition of features*[17].

The first stage of acquiring data involves collecting pictures, most commonly captured manually from one or multiple cameras due to the availability of video cameras in the modern-day, this is where pre-processing comes in handy for the fact that different types of cameras provide different quality images, creating noises that may interfere with the process of feature extraction, some of the commonly used techniques include Median and Gaussian Filter. Less common but also noteworthy is the use of the Kinect camera[18], [19], [20], with built-in depth sensors that can improve segmentation by providing more information about the background and foreground of the image, although this method is more expensive and less accessible for most people than a typical camera. Alternatively, the required data can also be obtained via publicly available datasets on the internet. Although most datasets created are of good quality and adequate amount, Data Augmentation can also be employed to create more if the number of provided data is not satisfactory, by using the pre-existing data to produce newer and unique data[18]. This method can help improve the accuracy of the model by reducing the risk of overfitting and offering the classifier more consistent data.

Extraction of features is the broad generalization of the next stage in which the most relevant parts of the input are reserved and heightened, these parts will then be fed into classification algorithms. These features vary depending on the research and the relevance of which information is important for which project. For instance, some researchers opted to explicitly separate the full mass of the hand from the background using a combination of motion, skin color, and edge detection[21], while others chose to focus only on certain points on the fingers instead. Most conventional extraction methods employ Convolutional Neural Networks (CNN) due to their robustness in categorizing hand gesture patterns without the need for segmentation. As the technology for feature extraction evolves, higher fidelity solutions for finger tracking appear, one prominent example is Mediapipe Hands[22], developed by Google. Instead of fingertips positions, Mediapipe Hands detect the palm and create a bounding box surrounding the detected area by operating with a Machine Learning (ML) pipeline, followed by a 3D landmark detector that accurately maps out

all 21 essential landmarks or key points corresponding to the joints on the hand, as seen in Figure 1. Simultaneously, the hand is cropped by the ML pipeline using the landmarks information provided by previous frames. Since its release to the public, many studies have been conducted in an effort to further understand the algorithm and apply it in hand gesture recognition applications. More recently, Sign Language Translation based on Mediapipe Hands has also been an active research topic, usually in combination with other types of Artificial Neural Networks (ANN) like CNN [23] or LSTM [24] for training and deployment, warranting an abundant of attention given its high reliability and relatively simple integration into other programs, lending great potential for further development and applicability. This leads to the final classification process or recognition of features, which offers an array of different methods one can choose to train their model, making extensive usage of the Neural Network. As previously mentioned, the most popular method is the use of CNN due to its accuracy, as exemplified by works such as [25] with translation accuracy reaching as high as 99.39%. What separates CNNs from the usual ANN is the extra convolutional layers that extract features and categorize data into their respective nodes while repeatedly learning from the results of the training, which gets better with more layers at the expense of heavier computations and strong hardware. There are several other alternatives with lower performance but require less computational power like the default ANN or Cross-Correlation Coefficient method as compiled by the research group in [5].

Although considerably advantageous when compared to its device-based counterpart, vision-based hand gesture recognition applications are not without their drawbacks. One of the main points being brought up constantly when arguments are made against Computer Vision-based technology is its restrictions in their space of operation, limited within the borders of the frames of the cameras, forcing the users to only maneuver their hands within a certain pre-determined area. Another concern raised was the vulnerability towards changing light conditions and non-uniform backgrounds, or self-occlusion problems when more sophisticated finger movements are present, causing difficulties in hand detection. Furthermore, the ratio between works dedicated to the research of solitary static postures compared to studies done that progress the state of detecting continuous movement of the hand/fingers is indicative of the inherent complexities of these gestures as well as the incapability of known models being able to identify them at an adequate precision, signaling that there is more work to be done before this concept is ready for on-site applications[8]. Some of these issues will be addressed in the next design segment for this project.

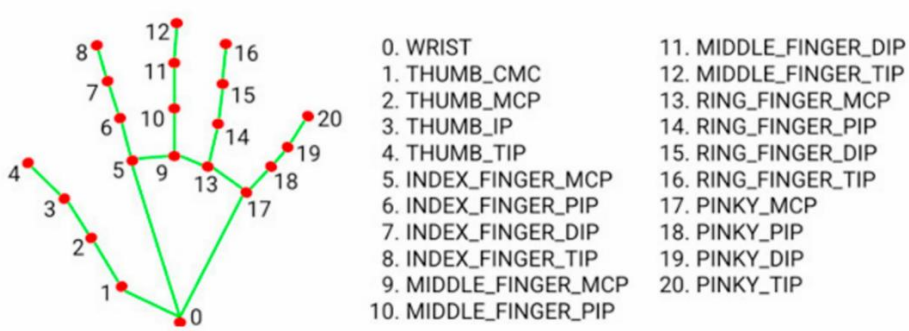


Figure 1: 21 hand landmarks localized by MediaPipe hand detection model (Source: [22])

2. Application Areas

The potential for hand gesture detection and recognition as a whole is undeniably vast, with numerous fields and services already planning and adopting the technology into their domains. Instances can be seen in Virtual Reality interactions, vehicle interfaces, robotics, and medical applications, with one of the most highlighted areas being Sign Language Translation and assistive technology for people who are unable to perform certain tasks due to bodily or cognitive limitations. Studies are being carried out daily on the importance of communicative gestures, sign languages as with any other languages is inherently structural, providing an ideal testing ground for hand sign detection algorithms, all while finding new ways to offer more inclusivity for people with disabilities. Whether it is directly translating a sign language user's ideas into texts or spoken words, allowing Human-Computer interaction between verbally ineffective or hearing impaired individuals and voice-activated Artificial Intelligent Assistants like Cortana, Siri, and other Smart Home applications, or simply providing an easy and accessible method for people to understand and get into learning sign languages themselves, hand gesture detection and recognition devices are undoubtedly beneficial not only towards the disadvantaged but for modern society as a whole. In conclusion, The future of practical applications of hand gesture recognition in day-to-day life is a good possibility now with active research being published as well as new advancements being made. However, many weak points of the system still need to be addressed in order for the technology to move forward and improve the capability of detection regardless of environmental factors. The next chapter will introduce a design idea that tackles some of the aforementioned drawbacks of the vision-based system by integrating hardware and sensor elements of the device-based approach.

Chapter 3: Methodology

1. Proposed System

The general concept of this sign language translator is to use image processing and sensors in combination to detect the hands, their orientation, movement, and environmental factors that can affect the translation process when performing the sign language, then the hand gesture data will be stored and processed by a neural network algorithm within the Raspberry Pi computer. This database stores each sign's characteristic finger positions as key point coordinates, associated with different articulations. Providing the position of each finger to detect which gesture is the hand making. Once a match is found, the program can trigger the output, which could be either a printed text of the translation on a screen or a text-to-speech corresponding audio associated with the recognized sign in the database. At the same time, a photoresistor sensor will be implemented to detect the light quality in the room, detecting when the ambient brightness is on lower levels and activating an array of LEDs on the device, illuminating the subject and enhancing detection visibility. The camera itself is attached to a Pan-and-Tilt setup that involves two servos controlled by the detection program through the use of an Arduino microcontroller, the job is to track the orientation of the subject's hands and use the two attached servos to position the detected hands in a way that the user is always within the frame of detection, addressing the issue of the restricted space. The established framework of Mediapipe will be applied to the dataset preparation and hand-tracking applications. The entire translation pipeline will include data collection, preprocessing, model training, classification, sensor integration, and finally system implementation. The proposed system for this translator is shown in Figure 2.

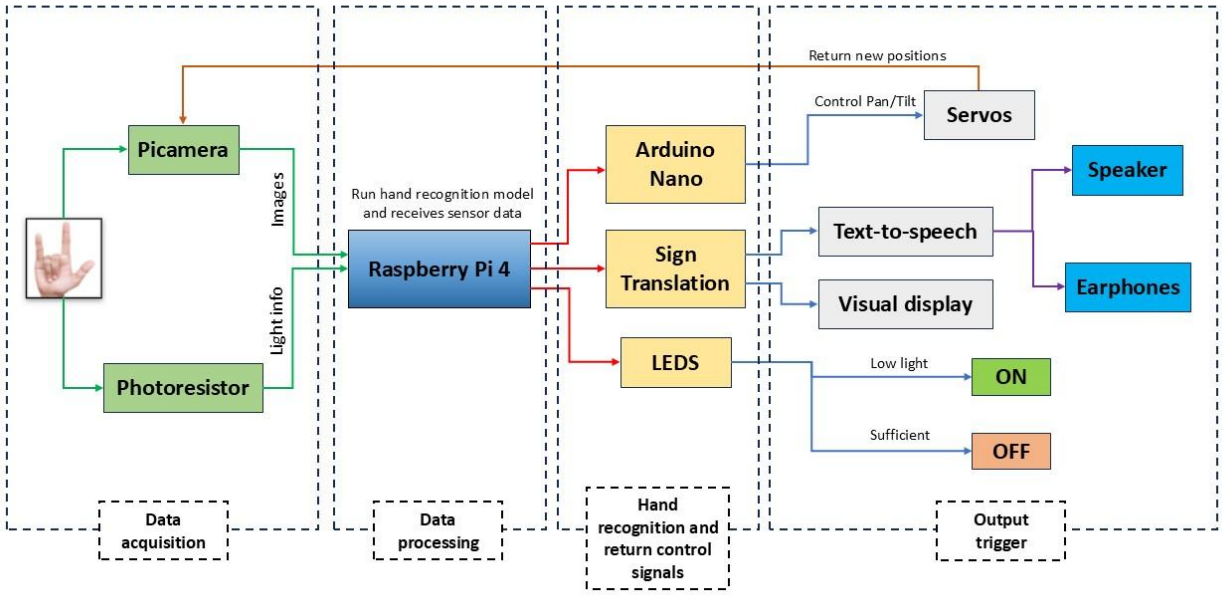


Figure 2: Block diagram of the processes and architecture of the system

2. Sign language system: American Sign Language system (ASL)

Figure 3: The American Sign Language alphabet (Source: Stock Images)

Even though there are various sign language systems currently used all over the world, this project focuses on developing a sign language translator specifically designed to recognize and translate American Sign Language (ASL) which is one of the most widely used sign languages in the world. American Sign Language (ASL) is a fully developed natural language, exhibiting linguistic properties comparable to spoken languages, and relies primarily on hand and facial movements for expression[26]. In this project, we focus solely on hand postures performing the ASL alphabet, a fundamental component of sign language communication, forming the basis for spelling words and proper nouns not readily expressible with hand signs. The ASL alphabet hand gestures are shown in Figure 3. The following sections will dive into the technical aspects of the device, including the program, the neural network, and the sensors used to capture hand movements, as well as the data analysis process for sign recognition.

3. Mediapipe

As briefly mentioned in Chapter 2, MediaPipe hand key point detection is fundamentally based upon a machine learning (ML) pipeline. One of the models in the MediaPipe ML pipeline executes palm detection on a frame and outputs a bounding box around the hand that is adapted to the varying angles of the captured image. A high-fidelity 3D hand landmark[23] detection is then applied to the region of interest, mapping out all 21 landmarks and cropping out the important hands using the information from the previous frame, if the hand landmarks detection is unsuccessful in the localization of the hand, the palm detection model will be of service. This project will make intensive use of Mediapipe frameworks, as well as the free and open-source software library of Tensorflow due to its vast applications in tasks such as training and inferencing

deep neural networks. The capturing and tracking of hand key points will be performed by Mediapipe while Tensorflow will be tasked with the training and recognition of the ML algorithm.

The dataset to be used in conjunction with the proposed system was collected with the help of Mediapipe using the open-source library's 21 key points hand detection feature, generating a dataset of coordinates that have been automatically normalized into trainable data, ready to be fed into Tensorflow. There will be 2 datasets, one containing 24 of the letters in the ASL alphabet that do not require movement from the hand itself and 2 for a separate set for motion-based detection. Each letter has 80 different permutations and is collected from 4 different humans, 2 males, and 2 females, making up for a total amount of values of 151,536, not counting the irrelevant parameters.

4. Sign recognition system description

The sign language translation device is designed to be multi-functional, user-friendly, relatively portable, and unobtrusive while capturing the necessary hand movements for accurate sign recognition. The device prototype is composed of small modules attached to a board, with a plastic casing for the main processing unit and pan-tilt components for hand-tracking applications. The sensors are mounted onto the pan and tilt component itself to record the environmental changes while hand signs are detected in real time. These sensors will be connected to a microcontroller (Arduino Nano) powered by the main computer (Raspberry Pi 4 Model B) which also drives the servo motors and processes the camera inputs. The utilization of an Arduino control allows for more processing power of the Raspberry Pi to be focused on the task of detecting and translating the hand signs. The audio output will be positioned within the system and can be accessed via a 3.5mm jack through a speaker or a headphone. The circuit connection between these components and the actual design will be discussed further in later sections.

4.1. Components

4.1.1. Raspberry Pi 4 Model B

The Raspberry Pi microcomputer is popular amongst software engineers and hobbyists due to its affordability and versatility. The single-board computer can be used for numerous applications, from developing hardware or software projects to employment in industrial settings and computer vision and object detection implementations. For this research, the Raspberry Pi 4 Model B with 4GB of RAM was elected for its high-performance 64-bit quad-core processor, dual-display support with resolutions up to 4K through two micro-HDMI ports, VideoCore VI 3D Graphics and hardware video decode at up to 4Kp60[27], beyond sufficient for effective hand detection. Although the specifications of the recently released Raspberry Pi 5 are undoubtedly superior to those of the Raspberry Pi 4, the availability and budget factors were brought into consideration during the course of this study, elements of which the Raspberry Pi 4 adequately satisfied. Another reason for the utilization of the Raspberry Pi computer in this project is its size and portability, which fulfilled another requirement of the prototype device. The mechanical specifications and components of the board are detailed in Figure 4 and Figure 5, respectively.

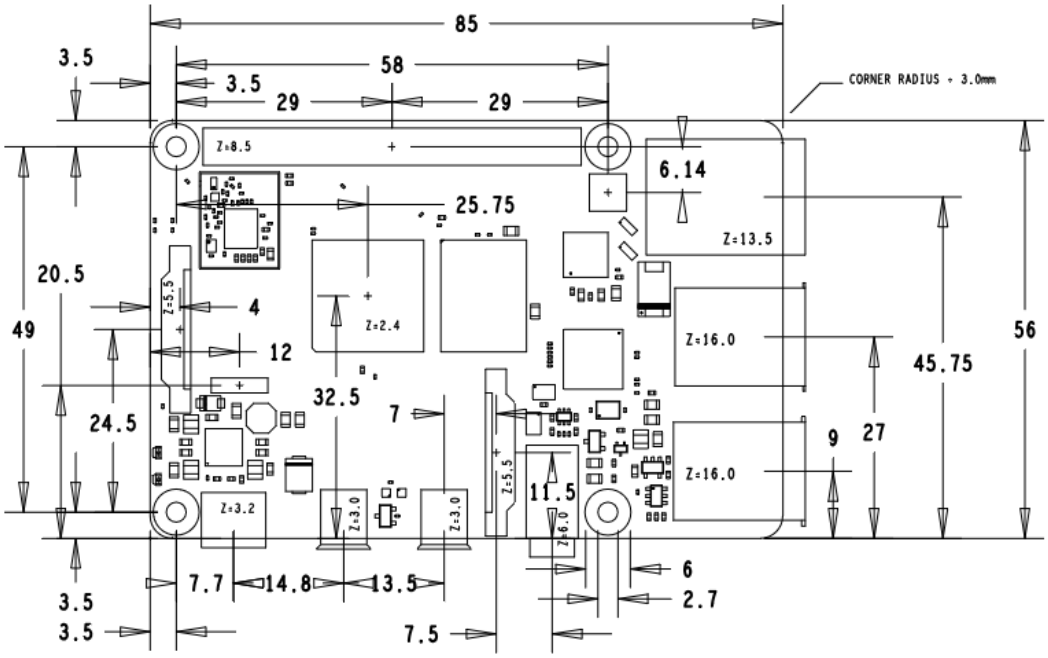


Figure 4: The mechanical specifications of the Raspberry Pi 4 (Source: [27])

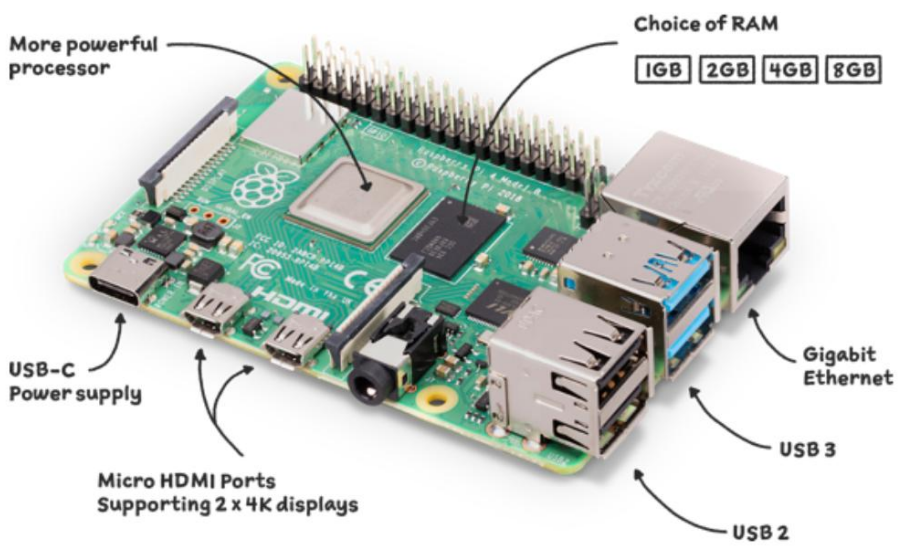


Figure 5: Raspberry Pi 4's essential components (Source: Raspberry Pi official website)

4.1.2. Raspberry Pi camera

For this project, the Raspberry Pi camera rev 1.3 module was employed for the task of capturing the image that will be processed by the main board for hand recognition. The module is an official product of the Raspberry Pi Foundation with full support of the software and hardware required to run camera applications. The camera module offers a 5-megapixel RGB sensor that supports 1080p30, 720p60, and VGA90 video modes[28]. The module's PCB is approximately 25x24mm in size[29] and they are connected to the Raspberry Pi via a replaceable flex cable of various lengths depending on the application. The module's measurements are illustrated in Figure 6.

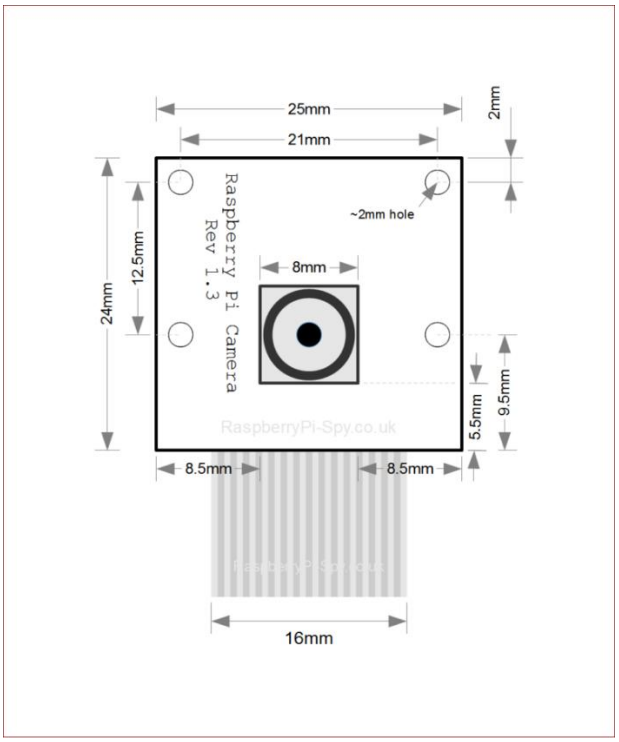


Figure 6: Mechanical specifications of the Picamera module (Source: [29])

4.1.3. Arduino Nano

Aside from the main task of hand detection and gesture recognition, sensors and servos will also be implemented into the design of the translator, to save on processing power, these secondary tasks will be performed by an Arduino microcontroller. Specifically in this study, the classic Arduino Nano model has been chosen due to its compactness in size and low power consumption while still maintaining all aspects that make the larger models effective. This board will be powered by one of the USB ports of the Raspberry Pi at a consumption rate of about 20 to 30 mA. The connecting pins and measurement information about the Nano[30] can be found in Figure 7 and Figure 8, respectively.

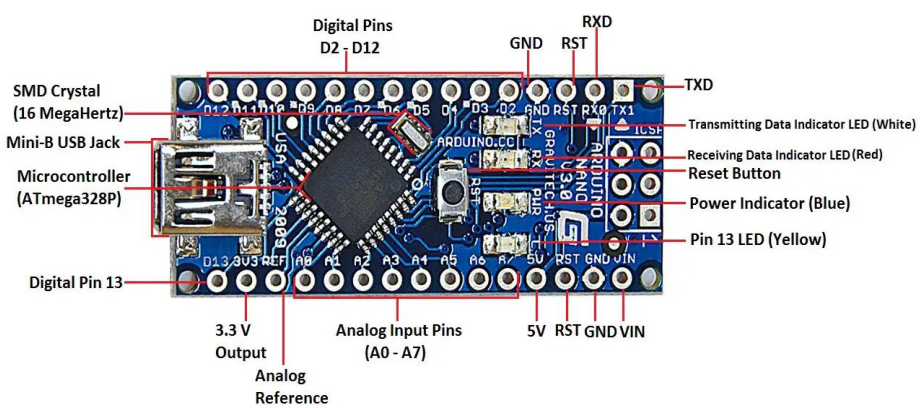


Figure 7: The components and pins of the Arduino Nano (Source: [30])

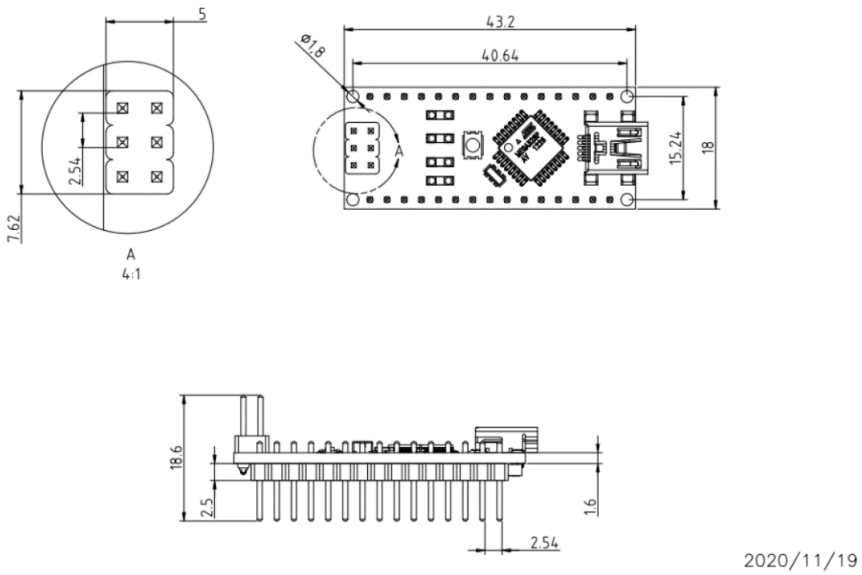


Figure 8: The measurements of the Arduino Nano (Source: [30])

4.1.4. Softwares

The two most important components that will handle the heavy computations and control other components are the Raspberry Pi 4 and the Arduino microcontroller, which require an Operating System (OS) and an Integrated Development Environment (IDE), respectively. The Raspberry Pi runs on the Raspbian OS and the Arduino IDE is used to develop codes that can be uploaded to the Arduino Nano to run repeatedly once it is powered.

4.1.4.1. Raspbian OS/ Raspberry Pi OS/ Linux

The Raspbian Operating System or Raspberry Pi OS is based on the Debian Linux distribution[31], it is free to download on the Raspberry Pi website. In order to start using the Raspberry Pi 4, a microSD card must first be flashed with the operating system using the Raspberry Pi Imager application. The microSD card works similarly to the disk drive within a typical computer or the flash memory in mobile phones, therefore the performance of the Raspberry Pi is greatly dependent on the performance of the card itself, it is crucial to make sure that the chosen memory card is of sufficient storage and is as fast and reliable as the tasks require the performance of the Pi to be, a 64Gb Samsung microSD card was used for this project. The imager offered many options for OS that can be chosen to be written into the memory card, for this particular case, a port of the 64bit Raspberry Pi OS (Debian Bookworm) was chosen as opposed to a more recent port of the Debian Bullseye as during the development of the detection program, there were encounters of errors with the software. The main hand recognition program itself will be run with Thonny, a Python IDE preinstalled with and supported by the Raspberry Pi OS.

4.1.4.2. Arduino IDE

Arduino IDE software is used to program the Arduino Nano with the codes that allow the main detection Python program running from the Raspberry Pi to communicate and control the Arduino's functions. This is achieved by employing Firmata[32], which is an intermediate protocol that connects an embedded system to a host computer, essentially making the microcontroller an

extension of the programming environment (Python in this case) on the host computer (The Raspberry Pi 4). By default, the protocol channel uses a serial port. The Arduino platform is the standard reference implementation for Firmata. The Arduino IDE comes with support for Firmata which can be accessed by navigating to *Examples* →*Firmata* →*StandardFirmata* in the Arduino IDE (only for version 1.6.X and later), the codes presented in this directory can then be exported and run on the microcontroller. Within the Python environment of the host machine, there are libraries that are required to implement the Firmata protocol after the Firmata firmware has been uploaded and is running on the Arduino Nano. PyFirmata is one of which can be installed via pip, providing a Python interface for the protocol to run on Python3, unfortunately, the original pyFirmata modules are currently only documented to operate on Python 3.7 and prior, which limits some of its functionalities for more recent Python versions. Given that for this project, the Raspberry Pi OS required comes with Python 3.11.2 readily installed in the system, it is preferable to look for another library that offers similar capabilities while also working effectively within more updated software. For this, we found that pyfirmata2, which is an updated version of pyFirmata and is still being actively supported, worked well for what this task is trying to achieve.

4.1.5. Photoresistor

A photoresistor or light-dependent resistor (LDR) is a simple and low-cost type of highly sensitive sensor that varies resistance values when in the presence of light, a higher intensity of light on its surface results in lower resistance and vice versa, which is suitable in applications that require an indication of the incidence or lack of light and measuring its intensity. For our purpose, an LDR Module[33] will be employed due to its availability and low price of $0.3 per piece. The module outputs a voltage in response to the changing resistance of the photoresistor, this voltage is available at the Analog Output (AO) pin. The same signal is then fed into a High Precision Comparator circuit for digitization and returned to the Digital Output (DO) pin. The module also includes a built-in potentiometer for sensitivity adjustment of the digital output (DO). For this project, we will mainly focus on the digital output of the LDR sensor since the two outputs LOW or HIGH will determine if the system's LEDS will be turned on or off, this threshold in which the signal switches from one state to the other can be adjusted using the potentiometer. The module also has two pre-installed LEDs. The Power LED indicates the module is powered while the Status LED signifies when the digital output goes LOW. The PCB module offers a complete and convenient package, making implementation within the system simpler. The LDR module's design with pins information and its circuit diagram are as detailed in Figure 9 and Figure 10, respectively.

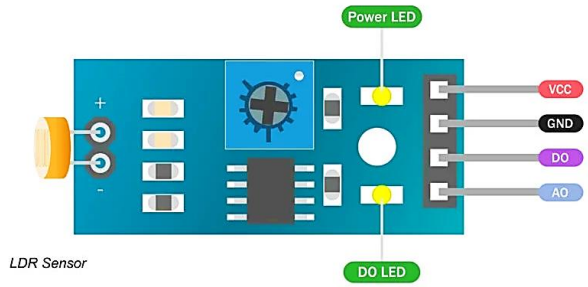


Figure 9: The components and pins of the photoresistor module (Source: [33])

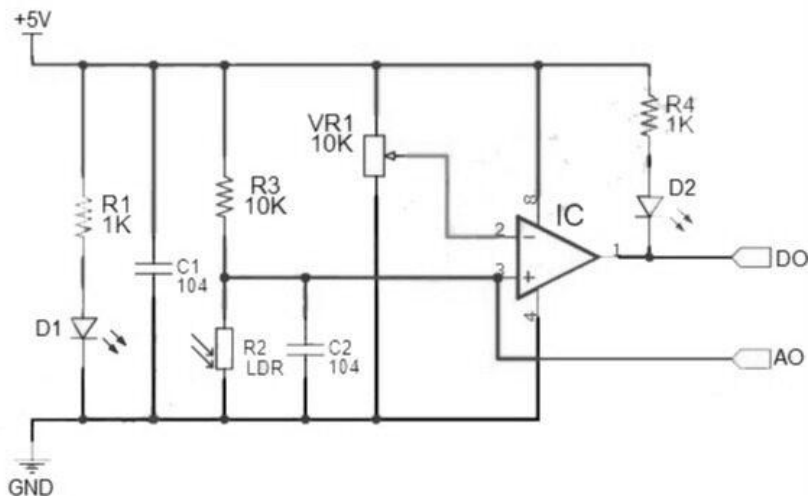


Figure 10: Electrical diagram of the photoresistor module, D1 is Power LED, D2 is Status LED (Source: [33])

4.1.6. Micro servo motors

For tracking purposes, we employ a mini pan and tilt unit coupled with two micro servo motors. Servo motors are useful when controllable speed and specific angles of rotations or positions are concerned in a project, usually applied in tasks involving robotic arms or steering systems. The most common micro servos on the market are the SG90[34] and MG90S[35] models, both offer the same range of rotation of 180 degrees, with similar affordability of around $1 each. One major difference is that the model MG90S comes with metal gears instead of plastic ones as seen in the SG90 model, ensuring better reliability and less chance of gear stripping. Coupled with the enhanced torque and speed, the MG90S servos are the preferred option for this project. Each of the servos comes attached with 3 wires corresponding to Signal (PWM), Power (VCC), and Ground (GND) in the colors of Orange, Red, and Brown, respectively. These wires will be connected to the Arduino Nano. The inner components of the servos are presented in Figure 11 along with the pan and tilt module.

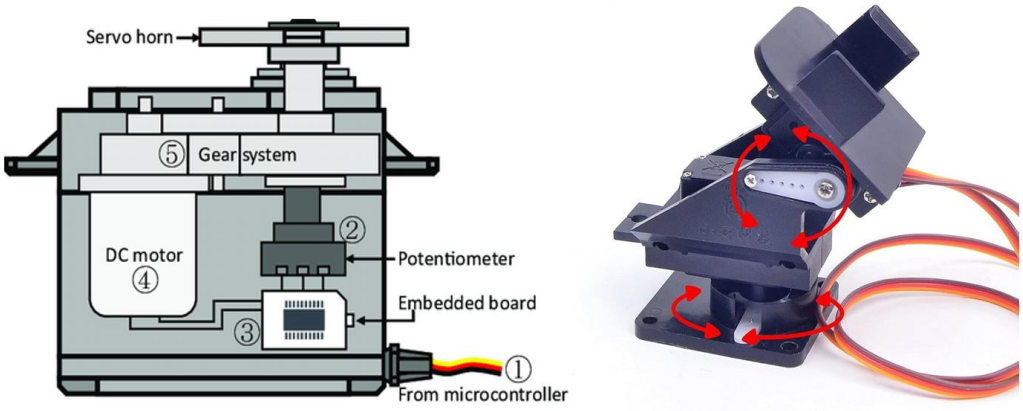


Figure 11: (left): The inner components of a microservo, (right): The pan and tilt unit and their rotational axes (Source: Internet)

The pan and tilt model comes with 4 separate pieces to be assembled into the complete unit. The "Pan" servo moves horizontally (azimuth angle) relative to the user while the "Tilt" servo moves vertically (elevation angle), Figure 11 also illustrates the workings of the Pan and Tilt mechanism. Although this can be adjusted, for this project, the ranges in which these Pan/Tilt servos will operate will be from 0 to 180 degrees for pan and 45 to 160 for tilt.

4.1.7. LEDs

A flat, circular module with 12 LEDs attached will be used for this study, the LEDs emit warm yellow light, which is found to be softer and less intense than white LEDs, helps reduce eye fatigue, and creates a more relaxed ambiance.

4.1.8. Speaker

The main audio output modes of the Raspberry Pi are 3.5mm headphone/speaker jack or USB-connected audio devices. HDMI audio output is also an option if a monitor is available to connect the Pi to, otherwise, either one of the first two options will suffice.

4.2. System architecture

The design of the hand-recognition device can be defined by two separate parts, each handling different aspects of the detection and translation process, one compensating for the drawbacks of the other. The software includes the hand detection and recognition program as well as the codes that communicate to the microcontroller the angles in which to position the servos that are tasked with keeping the tracking subject within the frame of the camera. The hardware involves the design and connections of the physical components that optimize the functions of the device while maintaining a clean and orderly appearance. The overall architecture of the neural network model can be seen in Figure 12.

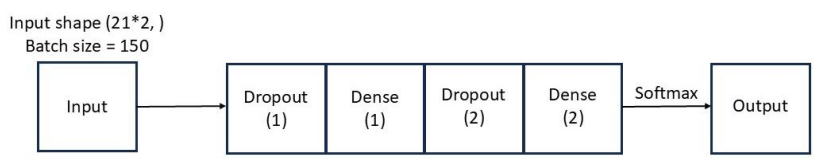


Figure 12: The architecture of the neural network

4.2.1. The Software

The process of developing a program that translates sign language postures involves three main stages: *Data acquisition, Training the program, recognition evaluation, and Raspberry Pi integration,* as illustrated in Figure. 11. The details of each stage in the development of the recognition software will be discussed in detail in the following paragraphs.

Figure 13: processes involved in developing the hand sign recognition model for Raspberry Pi

The foundation of the recognition program can be traced back to the Mediapipe Hands Library as documented in the previous sections. The Mediapipe gesture recognition employed a bundle of four different deep-learning models, as shown in Figure 12. The first model detects the presence and location of the hand by focusing on palm recognition, the second model then locates the 21 landmark locations on the hand including joints and fingertips (as seen in Figure), the third model maps the detected landmarks into numerical values (vector embedding) and the final step of determining the hand gesture is carried out by the fourth model. The advantage of using Mediapipe is the convenience of having these underlying details carried out by the algorithm in a user-friendly format that receives images and outputs values.

Figure 14: Mediapipe's process for hand detection and recognition

The Mediapipe hand detection model works optimally by lowering the frequency for the detection of the palm and only activates it if the landmark model fails, this reduces a significant amount of computation. This was accomplished by using the hand landmarks in the previously detected frame to derive the hand position in the current frames, neglecting the need to apply the palm detector for every frame. Additionally, the model also determines the confidence of the presence of a hand, when this confidence goes below a certain threshold, the palm detection model is reapplied to the next frame[22]. In Figure 13, the Mediapipe process for hand tracking is demonstrated.

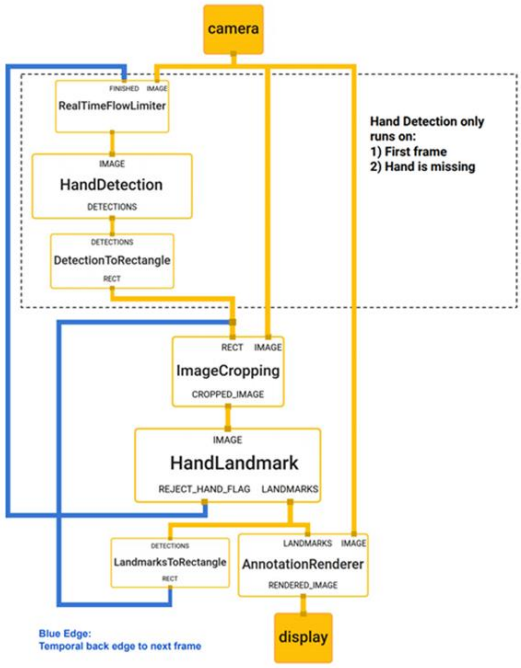


Figure 15: Mediapipe's graph for hand tracking, consisting of two subgraphs one for hand detection and another for landmarks computation, the landmark model output dictates when the hand detection model is activated. (Source: [22])

For a more detailed implementation of Mediapipe, the GitHub repository made by user Kazuhito00[36] was used as a basis to develop and train this project's hand recognition program. The repository included a sample program and two pre-trained recognition models, one recognizing simple hand signs and the other the moving gestures of the pointer finger, using a simple multi-layer perceptron (MLP) – a type of artificial neural network consisting of multiple layers of neurons, utilizing the landmark detection function provided by Mediapipe Hands.

The Data collection will be done within the program itself, by entering "K" while the detection window is running, the key point login can be initiated, and by pressing a number from 0 to 9 as the hand sign is visible within the camera frame, the coordinates of the hand can be recorded into a CSV file that can later be used to train the model. All 24 of the static signs in the ASL were captured, stored, and processed on an average laptop computer because once the models are ported into the Raspberry Pi computer, given its lower specifications, some reduction in framerate can be expected, therefore it is more ideal to develop the program on a more powerful machine and simplify it later for the Raspberry Pi. The required libraries needed to successfully run the test program and acquire data are detailed in [36], all the requirements can either be of the same version as detailed or one of the later releases.

For the purpose of developing the dataset, the test program will employ the built-in laptop camera. The adjustable options of the demo program including the width and height of the camera capture, and the minimum detection and tracking confidence will remain largely unchanged for the process of logging in key point coordinates, the only change to be made is the number of hands onscreen that the program is allowed to detect at once, this option will be changed to 2 since both hands will be providing information at this stage.

The acquisition of the dataset involves the collecting of hand landmarks from 4 subjects, 2 males and 2 females, each providing 20 iterations for each recorded hand sign in various lighting conditions and distances, ensuring good variety for the training section. Noted that although these measures were taken with the intention of increasing the accuracy of the model once training is completed, they are not necessarily required and their effects on the results overall may be minimal due to the already highly-processed, high-fidelity hand detection module provided by Mediapipe, which have accounted for all the previously concerned accuracy issues by employing a precise and thorough method of combining real-world and synthetic datasets, collecting data in-the-wild, in-house and with synthetic hand models that varies in textures and skin tones in random high-dynamic-range lighting environment and different cameras[37]. This results in a highly robust hand recognition model that can detect hands even in shadowy and lowly-lit areas. The process of extracting the landmarks data from the hand with the hand recognition window using the Logging Key Point function can be seen in Figure 16, it can be seen that although the hand region in this picture is opposite to the light source, casting a shadow region into the palm, the model remains efficient in the bounding and mapping of the key points on the hand.

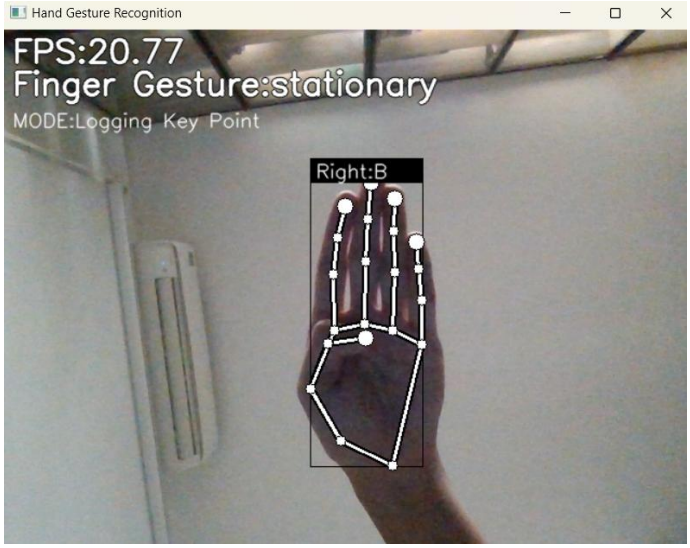


Figure 16: Video capture of hand recognition during data collection of key point coordinates

After each letter's data is collected, their respective data will be pre-processed (normalized) and stored in the *model/keypoint_classifier/keypoint.csv* directory, along with the CSV file for each of the labels that will be associated with the signs themselves. One issue with the original repository's program is its inability to record more than 10 different hand signs, this was remedied by adding and changing several lines of codes within the sample inference program, allowing the key point logging function to operate above the limited recorded amount of 10 hand postures. The way the key points are presented in the CSV file and the normalization process in which the pixel

Table 1: First column: order of sign, Second and subsequent columns: landmark coordinates

0	0	0	-0.41584	-0.0099	-0.78218	-0.27723	-0.9505	-0.62376	-1	-0.89109	-0.66337	-0.71287
0	0	0	-0.41	-0.02	-0.78	-0.3	-0.95	-0.64	-1	-0.91	-0.66	-0.73
0	0	0	-0.37226	-0.0219	-0.68613	-0.27007	-0.83212	-0.54015	-0.93431	-0.79562	-0.66423	-0.86131
0	0	0	-0.41509	-0.04717	-0.73585	-0.38679	-0.83019	-0.74528	-0.85849	-1	-0.57547	-0.72642
0	0	0	-0.38	-0.1	-0.67	-0.44	-0.76	-0.77	-0.79	-1	-0.48	-0.76
1	0	0	-0.15106	-0.0574	-0.23263	-0.20544	-0.13293	-0.34139	0	-0.35045	-0.25982	-0.44109
1	0	0	-0.14869	-0.04956	-0.24198	-0.1895	-0.13994	-0.3207	0	-0.34111	-0.26531	-0.42857
1	0	0	-0.15951	-0.04908	-0.26687	-0.18712	-0.18098	-0.32209	-0.04294	-0.34356	-0.28221	-0.43252
1	0	0	-0.15891	-0.05426	-0.26744	-0.1938	-0.18605	-0.32558	-0.05814	-0.36047	-0.28682	-0.44186
1	0	0	-0.15487	-0.06195	-0.26549	-0.20354	-0.18584	-0.33186	-0.04867	-0.34071	-0.28761	-0.45575
2	0	0	-0.31176	-0.01765	-0.6	-0.09412	-0.82353	-0.08235	-1	-0.14706	-0.53529	-0.54706
2	0	0	-0.29885	-0.03448	-0.59195	-0.13793	-0.82759	-0.16092	-1	-0.22414	-0.5	-0.59195
2	0	0	-0.3198	-0.0203	-0.58883	-0.11675	-0.81726	-0.13706	-1	-0.22843	-0.46701	-0.58376
2	0	0	-0.29032	-0.01382	-0.58525	-0.1106	-0.82028	-0.12442	-1	-0.18433	-0.48387	-0.58065
2	0	0	-0.2963	-0.02778	-0.59259	-0.13426	-0.82407	-0.14352	-1	-0.19444	-0.49537	-0.58333
3	0	0	-0.1435	-0.04484	-0.25112	-0.10314	-0.2287	-0.12556	-0.13453	-0.14798	-0.21076	-0.46188
3	0	0	-0.15126	-0.04622	-0.26891	-0.10924	-0.2563	-0.13866	-0.15546	-0.16387	-0.20588	-0.45378
3	0	0	-0.14463	-0.04132	-0.26446	-0.09504	-0.26033	-0.12397	-0.16116	-0.15289	-0.20661	-0.45868
3	0	0	-0.17204	-0.04839	-0.3172	-0.13441	-0.35484	-0.22581	-0.26344	-0.2957	-0.24731	-0.51075
3	0	0	-0.16484	-0.05495	-0.28022	-0.14835	-0.28022	-0.23077	-0.21429	-0.3022	-0.22527	-0.51099
4	0	0	-0.3	-0.03077	-0.59231	-0.28462	-0.55385	-0.53077	-0.28462	-0.52308	-0.49231	-0.70769
4	0	0	-0.30657	-0.0292	-0.59854	-0.28467	-0.54745	-0.51825	-0.27737	-0.51095	-0.49635	-0.70803
4	0	0	-0.29412	-0.03922	-0.56863	-0.29412	-0.50327	-0.52288	-0.22876	-0.49673	-0.45752	-0.72549
4	0	0	-0.29341	-0.03593	-0.57485	-0.28144	-0.51497	-0.52096	-0.23353	-0.50299	-0.47904	-0.74251
4	0	0	-0.29412	-0.01176	-0.61176	-0.25294	-0.57647	-0.5	-0.29412	-0.49412	-0.52353	-0.72353

coordinates of those key points are transformed into trainable data ranging from -1 to 1 can be seen in Table 1 and Table 2, respectively.

Table 2: The normalization process of landmark coordinates

(1) Landmark coordinates					
ID: 0	ID: 1	ID: 2	……	ID: 19	ID: 20
[551,465]	[485, 428]	[439,362]	……	[687,225]	[702,188]
(2) Convert to relative coordinates from ID: 0					
ID: 0	ID: 1	ID: 2	……	ID: 19	ID: 20
[0,0]	[-66,-37]	[-112,-103]	……	[136,-240]	[151,-277]
(3) Flatten to a one-dimensional array					
ID: 0	ID: 1	ID: 2	……	ID: 19	ID: 20
[0 0]	[-66 -27]	[-112 -103]	……	[136-240]	[151 -277]
(4) Normalized to the absolute value					
ID: 0	ID: 1	ID: 2	……	ID: 19	ID: 20
[0 0]	[-0.24 -0.13]	[-0.4 -0.37]	……	[0.491 -0.87]	[0.545 -1]

For the training part of the process, we relied heavily on the use of Jupyter Notebook, formerly known as Ipython Notebook, an interactive open-sourced web application. This tool offers a flexible environment for executing the training codes in separate sections, creating data visualization, and configuring easy-to-follow, intuitive workflows for data science and other machine-learning purposes. The training program is contained within the file *keypoint_classification.ipynb* can be opened via the Terminal command Jupyter Notebook. Inside, the number of data classes is defaulted at 3, this is changed depending on how many hand signs were collected, in this case, we are looking at 24 static postures for the 24 alphabetical letters.

After importing tensorflow and all the other necessary libraries, the random seed is established in order for the training process to be reproducible on other machines by generating the same random numbers on different executions, and then the paths to the dataset and the location in which the finished model will be saved. The dataset location is saved within the *model/keypoint_classifier/keypoint.csv* directory while the model save path is *model/keypoint_classifier/keypoint_classifier.hdf5*, the hdf5 extension is used to save and load the machine learning model in a Hierarchical Data Format (HDF5) due to their ability to store large amounts of data[38]. X and y sets are defined from the dataset as the input samples and their corresponding characteristics. The two sets are then ready to be split into two separate sets each for training and testing, the size of the training set by default is set as 75% of the original data. After everything is established, the model-building process is commenced. For this project, we are applying the sequential model to create a Feedforward Neural Network with five layers including two dropout layers and three dense layers, the summary of the model with the layer type, output shape, and parameters can be found in Figure 17.

```
Model: "sequential_6"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 dropout_12 (Dropout)        (None, 42)                0

 dense_18 (Dense)            (None, 20)                860

 dropout_13 (Dropout)        (None, 20)                0

 dense_19 (Dense)            (None, 10)                210

 dense_20 (Dense)            (None, 24)                264

=================================================================
Total params: 1,334
Trainable params: 1,334
Non-trainable params: 0
_________________________________________________________________
```

Figure 17: The model summary of the neural network

Backpropagation is the process carried out by multi-layered perceptrons, feeding the output through the layers of the neural network, calculating the error between the predictions and the results, and adjusting the algorithm weights accordingly for more precise predictions[39]. ReLU activation or Rectified Linear Unit is applied to the first two dense layers of the system, which is an activation function that passes positive values directly as is and converts negative inputs into zero instead of allowing the neuron to be activated, lessening the computation and negating the vanishing gradient problem (as seen with the Sigmoid or tanH activation) in which the gradients used to measure changes in weights with respect to changes in error and update the neural network accordingly, reduces into increasingly smaller sizes as they are backpropagated from the output into previous layers. The activation function is important for determining whether the neurons will make a linear or nonlinear decision, transforming the input signal of a node in a non-linear way (as most real-world data are non-linear) into an output that can be passed to the following layer, allowing the neural network to learn complex patterns[40]. Due to its generally light computation while yielding better performance, the ReLU activation is often the default activation function for many kinds of neural networks. The ReLU activation is pictured in Figure 18 and is defined in Equation 1:

$$f(x) = x^+ = \max(0, x) = \frac{x + |x|}{2} = \begin{cases} x & \text{if } x > 0, \\ 0 & \text{otherwise,} \end{cases}$$

Equation 1

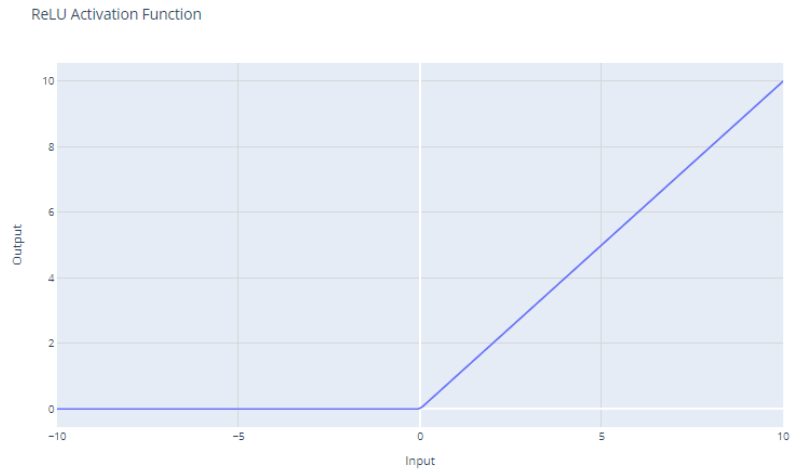


Figure 18: The graph for the ReLU Activation Function (Source: [40])

The last output layer which is also a Dense layer of the model utilizes the softmax activation or the normalized exponential function. This function is particularly useful for problems involving the classification of multiple classes, one which is similar to what this project deals with. The vector input on which the function operates embodies the scores for each class given by the previous layers. The output of the softmax function is a distribution of the probabilities of the classes to which each of the input values belongs (Figure 20), the use of the exponential function ensures no output values are negative due to them being probabilities. Softmax is typically used for its ability to produce probabilities, this can then be interpreted as a confidence score for each class and the model's predictions' certainty[40]. The Softmax activation is seen in Figure 19 and is defined in Equation 2:

$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} .$$

Equation 2

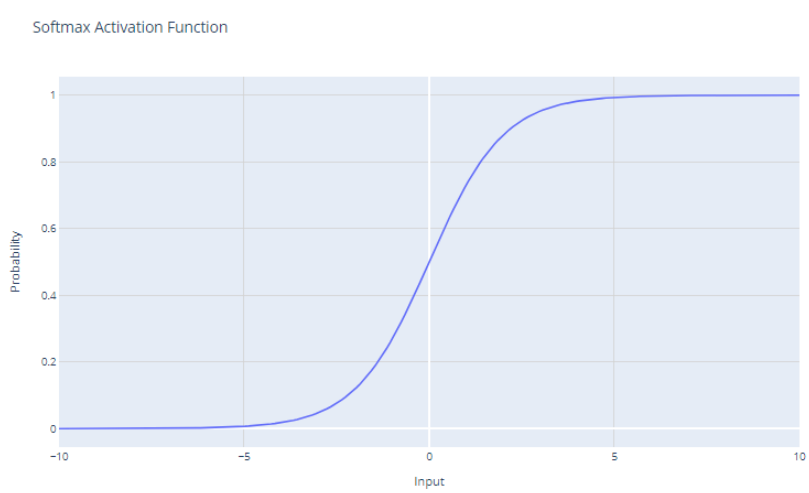


Figure 19: The graph for the Softmax Activation Function (Source: [40])

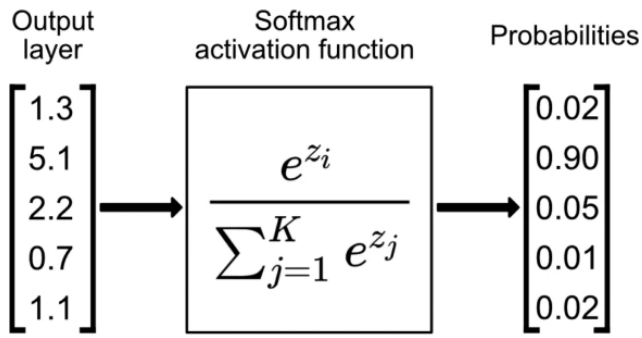


Figure 20: Input and Output of the Softmax Activation Function (Source: [41])

The system also employs the Adam Optimizer technique or Adaptive Moment Estimation. An optimizer works by trying to reduce the loss function by updating the parameters of the model depending on the output of the loss function (the difference between the predictions and the real values), increasing the model's accuracy. The optimizer accomplishes this task by updating the weights inside the algorithm, all optimizers follow the same formula for the weight update as shown in Figure 21. The Adam optimization technique will be used to update the network weights repeatedly. As opposed to the Stochastic Gradient Descent optimizer where the learning rate (α) remains constant throughout all weight updates, the Adam optimizer will compute individual adaptive learning rates for different parameters (including weights and biases, denoted as θ) at every iteration based on its gradient (g) history, maintaining a constant global learning rate while optimizing the speed of convergence and efficiency of the neural network as a whole. A gradient measures the change in all weights with respect to the change in error, it can also be imagined as the slope of a function. The higher the gradient, the steeper the slope gets and the faster the learning rate of the model is, consequently, if the slope is zero, the model ceases to learn[42]. All these specifications qualify Adam as the most robust and commonly used optimizing method for improved training speed in deep neural networks.

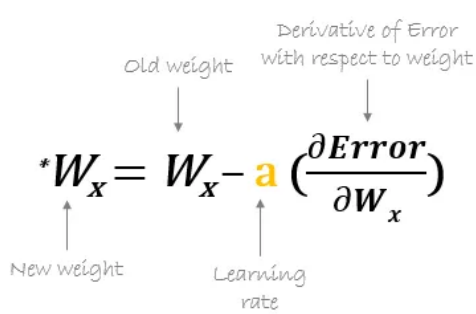


Figure 21: The formula for calculating new weights in an Optimizer (Source: [43])

The reason for the employment of the Feedforward or multi-layer neural network model is its straightforward architecture, in a Feedforward neural network, the nodes from the previous layer connect directly to the ones from the next layer, and data flows forward without going in loops or

cycles, this creates simplicity for further implementation and faster computation as well as requiring no memory due to not having to store the previous states. Although more advanced network designs such as CNNs or LSTM could and have been applied to similar models with sufficient results, such designs are unnecessary in the context of this project, with the most complex processing portion of hand detection and recognition has already been accomplished by the Mediapipe model, this allows for a lighter, less computationally expensive machine learning model to be employed for the training of the sign translation task. During the passing of data from one layer to the other, input nodes receive the data which flows through the hidden layers and leaves through the output nodes. The layers of a Feedforward Neural Network are illustrated in Figure 22. The input layer's neurons receive input data and pass it into the hidden layers, which are responsible for the learning of the algorithm. The number of hidden layers depends on the model, the model in this project has three hidden layers as seen in Figure 17. The hidden layers' neurons transform the input using the constantly updated weights (valued between 0 and 1) and activation functions.

Figure 22: The layers and parameters of a Feedforward Neural Network

There are two main functions that work within a Feedforward Neural Network: *Loss function and cost function.* The main purpose of training is to minimize the loss function, also known as the error function. This function measures the effectiveness in the performance of the neural network, its value is minimized to an acceptable degree during backpropagation. The loss function helps determine a weights and biases adjustment that enhances the performance of the model by quantifying the difference between predictions and true values. The minimum value of the function is found by changing the parameters during training, this value is then accompanied by the corresponding set of weights and biases that is optimal for making predictions[44]. A loss function applies to a single training example and is included in the model's overall learning process, providing the signal needed to update the parameters. In contrast, the cost function is essentially an average of the loss function of the whole training set, containing more than one example. Figure 23 demonstrates the logic algorithm of a loss function.

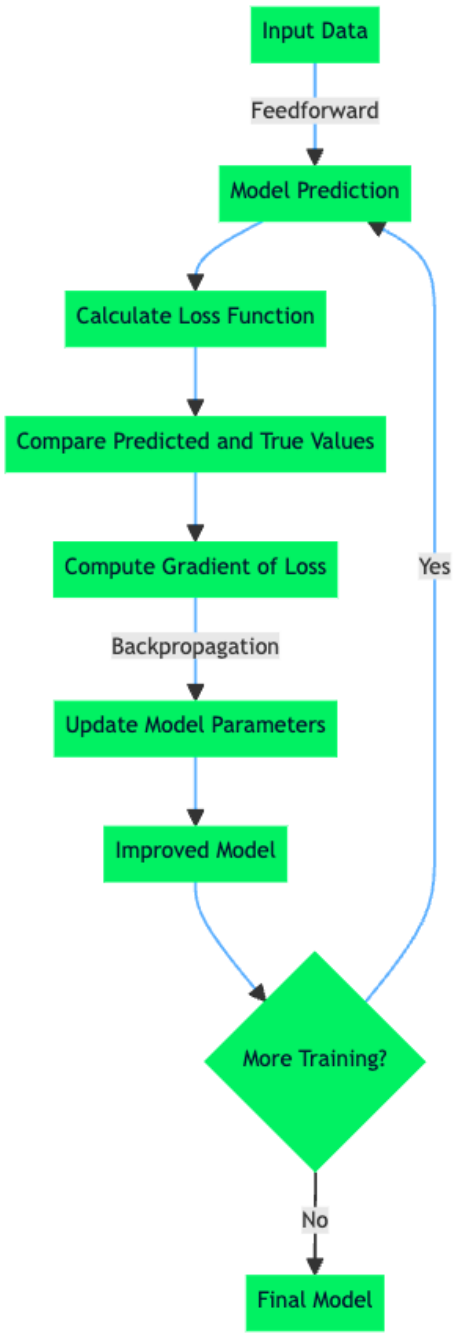


Figure 23: The logic algorithm of a Loss Function (Source: [44])

Although there are many types of loss functions, they all share the same fundamental operation. For this project, we are compiling the model by using a variant of the Cross-Entropy Loss, the Sparse categorial Cross-Entropy Loss, which is highly applicable for multi-class classification purposes. Cross-Entropy loss, logarithmic loss, log loss, or logistic loss where a comparison between the individual probability of the predicted class and the real class is made, loss is calculated to penalize the probability based on how inaccurate it is from the expected value. The penalty is logarithmic, for large differences close to 1 the score yielded is large, while a small score

for small differences tends to 0. A perfect model would have a cross-entropy loss of 0. The Cross-Entropy Loss formula is as follows in Equation 3[45]:

$$L_{\mathrm{CE}} = -\sum_{i=1}^{n} t_i \log(p_i), \text{ for n classes,}$$

where t_i is the truth label and p_i is the Softmax probability for the ith class.

Equation 3

Categorial Cross Entropy or Softmax Loss, is essentially a softmax activation function combined with a Cross-Entropy loss[46]. Categorical Cross-Entropy and Sparse Categorical Cross-entropy share the same loss formula as defined above, the main difference lies in the definitions of the labels. Categorial Cross-Entropy deals with one-hot encoded labels (ie. [1,0,0], [0,1,0], and [0,0,1] for a 3-class classification problem) while Sparse Categorial Cross-Entropy is used when labels are integer encoded (ie. [1], [2] and [3] for a 3-class problem) The extended formula for the Categorial Cross-Entropy Loss can be defined as presented in Equation 4:

$$H(y, \hat{y}) = -\sum_{i} y_i \log(\hat{y}_i) H(y, \hat{y}) = -\sum_{i} y_i \log(\hat{y}_i) H(y, \hat{y}) = -\sum_{i} y_i \log(\hat{y}_i)$$

Equation 4

Where y is the true probability distribution and ŷ is the predicted probability distribution (output of softmax). The sum of the product of the true label and the logarithm of the predicted probability for each class are calculated and then the result is negated. When the model's predictions are confident but incorrect the loss is high, low when the model's predictions match the true labels. This encourages the model to make accurate and confident predictions[47].

Within the model, there are two variations of layers inside the hidden stages, the Dropout layers and the Dense layers. The Dropout layers are a common method of preventing overfitting, which is when the algorithm fits too closely to the training data but cannot make accurate predictions for new data. This happens when the size of the training data is not of sufficient amount or diverse samples relative to the number of parameters and complexities of the model. The Dropout layers prevent this from happening by dropping a predetermined percentage of nodes (determined by a certain probability p from a Bernoulli distribution) from the hidden layer in every iteration (batch), minimizing the issue of co-adaptation. Co-adaptation is when in the process of forward propagation (training), the changing of neurons to decrease loss function and correct the errors made by other neurons leads to ineffectiveness in the generalization of the model. The dropout forces a more probabilistic approach onto the layers instead of relying on the units (neurons) to fix the mistakes of other units, allowing for a more generalized model, and alleviating overfitting[48]. In the context of this project's model, the probabilities of nodes randomly getting temporarily

removed are p1 = 0.2 and p2 = 0.4 for the first and second dropout layers, respectively. Figure 24 demonstrates the effects of adopting the dropout method:

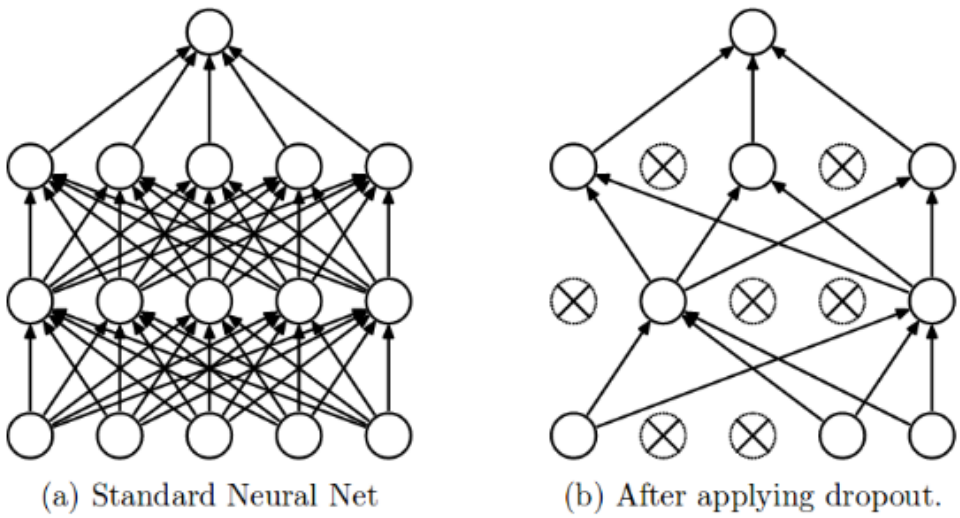


Figure 24: A neural network with and without the application of dropout layers (Source: [48])

Dense Layers on the other hand, instead of sparsing the connections and thinning the architecture of the model, connect deeply with the previous layers, every neuron of this layer is connected to the neurons presented in the preceding layer. The dense layer is the most common and basic type of layer in ANN, often used in the final stages of the neural network to preserve important features for the output layer. This layer has the benefit of being able to capture complex patterns in the data, although suffers from the number of parameters required and can easily be subjected to overfitting. In this model, these problems are remedied by the insertion of the dropout layers in between the dense layers. This is a common complementary combination in which sparse layers (dropout layers in this case) are paired with dense layers to achieve the best results. The sparse layer lessens the dimensionality of the input data that are then fed into the dense layers to capture more intricate patterns, improving the classification capability of the model[49].

After establishing the elements with which the model will be working, the training session is commenced. For this, a batch size of 150 and 400 epochs will be employed. An epoch is a composition of numerous iterations or batches, each iteration has a batch size, which is the number of sample data used to train the model in that iteration. The larger the batch size the more data the model processes at once, requiring more memory but cutting down the training time, while a smaller batch size offers more robustness but is also noisier, a batch size of 150 belongs in the prior category. Typically, the model's accuracy improves with each epoch but will cease learning once there is convergence, a number too small leads to underfitting as the model fails to learn complex patterns, and a number too big creates the risk of overfitting. Whether an epoch size is too much or too little is relative to the size of the dataset and the level of complexity of the task required for the model to perform. For hand detection and sign recognition, a larger number of epochs are usually needed to demonstrate better accuracy. For this study, a variety of epoch

numbers were tested with the model training to see when the performance hits a plateau, the number of epochs that was settled on that gives superior training results was 400. The model will go through the total dataset 400 times with each iteration processing 150 samples (batch size=150) before the model is updated, totaling a value of 60,000 batches for the entirety of this training process. Although possessing a considerably large number of epochs, the model trains relatively fast in under a minute.

After the training is completed and the classification yields the desirable results, the program can then be exported from the computer on which it was trained and imported into the Raspberry Pi. However, this process is not a linear one. In order for the recognition to work within a largely different software of the Raspberry Pi 4, many adjustments need to be considered. Firstly, the Raspberry Pi computer does not have a built-in webcam, instead, in order to capture images for detection, a picamera module is required. After the installation of the module onto the Pi, sections of the Python codes for the model need to be changed or added to start using the picamera for hand recognition. The required libraries are then installed onto the Raspberry Pi using sudo commands (granting security privileges as the superuser) and pip3, although in order to use pip3 commands to install packages on the Pi first the user needs to remove the externally managed restrictions in the default Python 3.11 on the Raspberry Pi, using the command:

sudo rm/usr/lib/python3.11/EXTERNALLY-MANAGED

After these setup stages, the installation of all the requirements is forward, although at first many issues concerning the packages were present, for example, the difficulty in installing OpenCV in some Raspberry Pi OS, or the inability of the Tensorflow installation to proceed with generating parts or importing certain packages included inside the library (ie. failure in building wheel for h5py), the fixes for these issues were later found by researching, looking for people sharing similar difficulties from online forums or change the method entirely. Another problem the project faced when attempting to run the model on the Raspberry Pi is one concerning the PyFirmata software. As briefed previously in the components section, PyFirmata offers solutions to the servo tracking problem by allowing communication between the Arduino microcontroller and the Python program on the Raspberry Pi. At the same time, the model runs smoothly on the computer it was trained on due to the installed Python being of an older version, the Raspberry Pi's more recent, updated Python 3.11.2 was experiencing trouble in this process. There were two ways in which this issue can be remedied, one is downgrading the Raspberry Pi's Python version to one of the previous releases, this method, as it went, was not an ideal solution for its complications in execution and a new set of issues regarding the library it brought. Eventually, it was found that a newer, more up-to-date port of PyFirmata called pyfirmata2 was released and is still currently being supported, this solved the problem of incompatibility with the most minimal adjustments required. Furthermore, the required pyfirmata codes were then uploaded directly into the Arduino Nano. The servo control programs are then written directly on the Thonny IDE on the Raspberry Pi and later integrated into the main hand detection codes. The servos are instructed to change their angles based on the return coordinates of a single landmark on the hand so that this specific landmark is always as centered on the frame captured by the camera as possible. Similarly, the codes to run the lighting unit are the most straightforward, the Pi receives information detected by

the photoresistor and based on whether there is enough light hitting the sensor, decides to trigger the LED on or off, illuminating the detected area.

After all the libraries were installed, the model demonstrated accurate detection and recognition capability while running on the Pi 4, although the framerates have reduced significantly from how it was performing on laptop hardware due to the limitations of the Raspberry Pi. This being said the integration of the model onto the small single-board computer for better portable applications was a success.

4.2.2. The Hardware

Using the circuit and PCB design software KiCad, the wiring schematics of the entire system are illustrated in Figure 25 and Figure 26. The hardware of this project is composed of a Raspberry Pi 4B, an Arduino Nano microcontroller, a photoresistor module, a Raspberry Pi camera module, two micro servos, a pan and tilt unit, and a lighting module connected to 12 mini LEDs, here for simplicity purposes, denoted as one single typical LED.

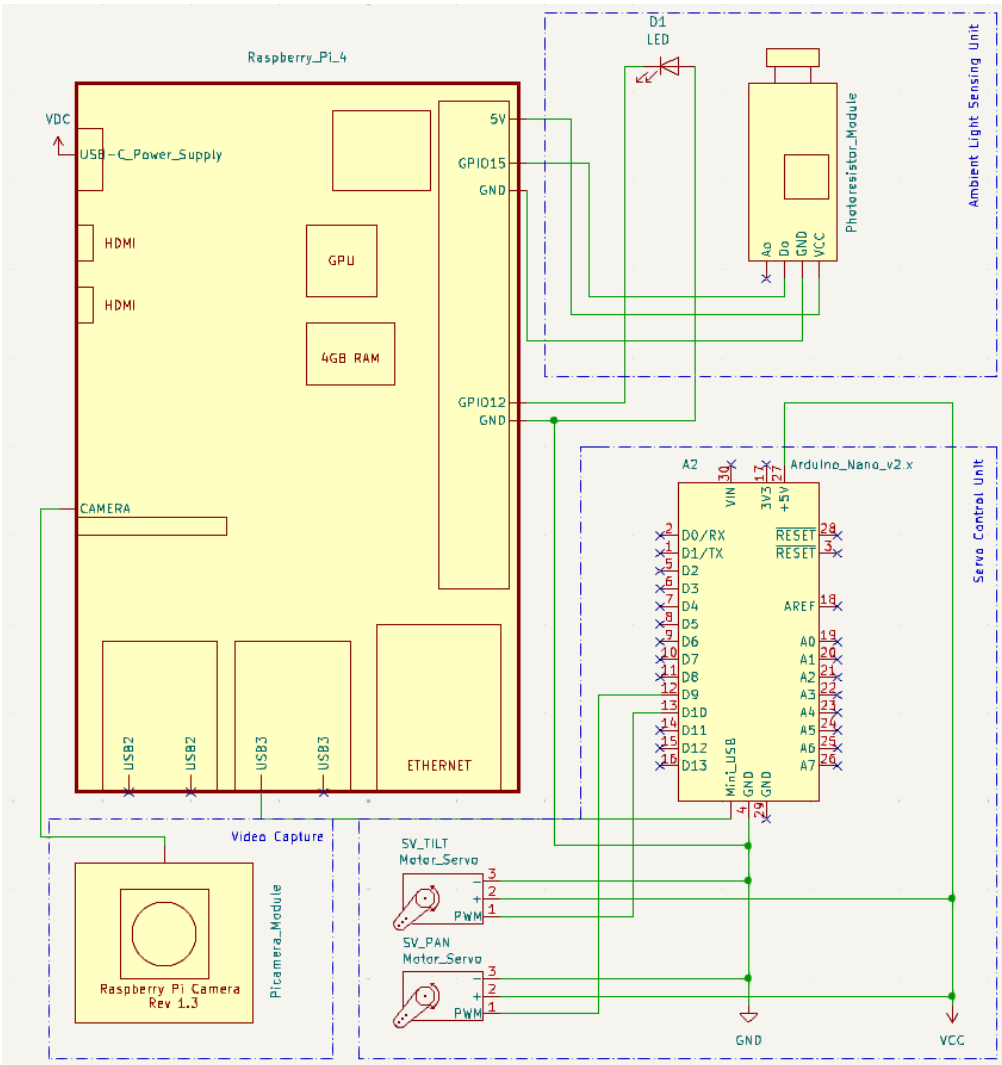


Figure 25: A schematic of how the components are wired together

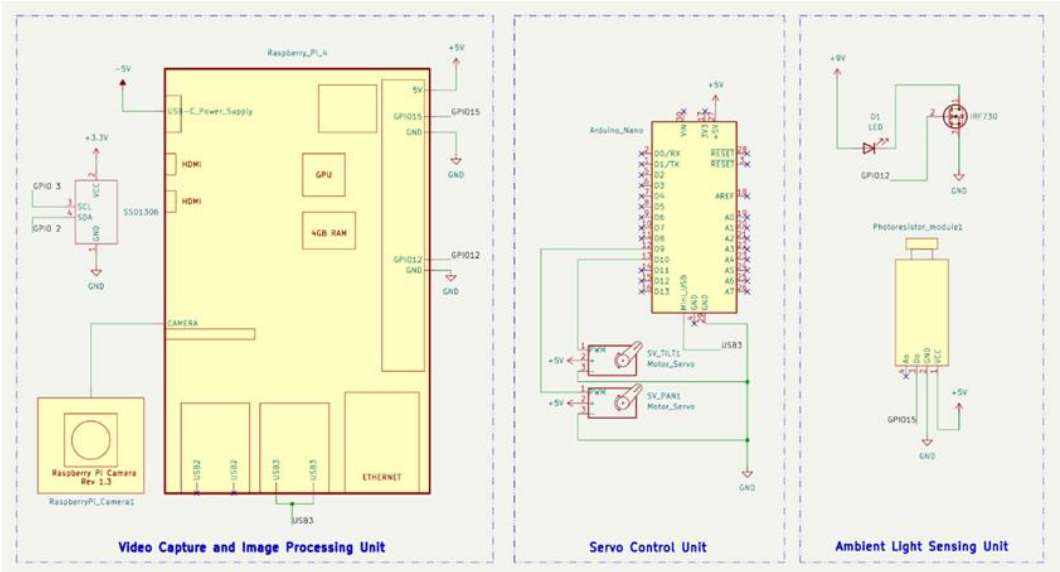


Figure 26: Hardware schematics, simplified

In this configuration, the Raspberry Pi 4 handles the majority of the computation, controlling the outputs of both the Arduino (and in turn, the pan and tilt servos) as well as the activation of the LEDs, and most importantly, the hand recognition software which will be displaying its output in either a visual or audio format. Figure 27 shows the design of the prototype device. The LED module will be controlled using a transistor as a digital switch and powered by an additional 9V battery due to the lights requiring more power than the output voltage the Pi offers. The pan and tilt unit is attached to the main case containing the Raspberry Pi 4 via thin magnetic strips, which should not interfere with any of the board's functions due to the Pi not using magnetic storage and the magnets not being strong enough to produce any discernable electrical currents inside the Pi.

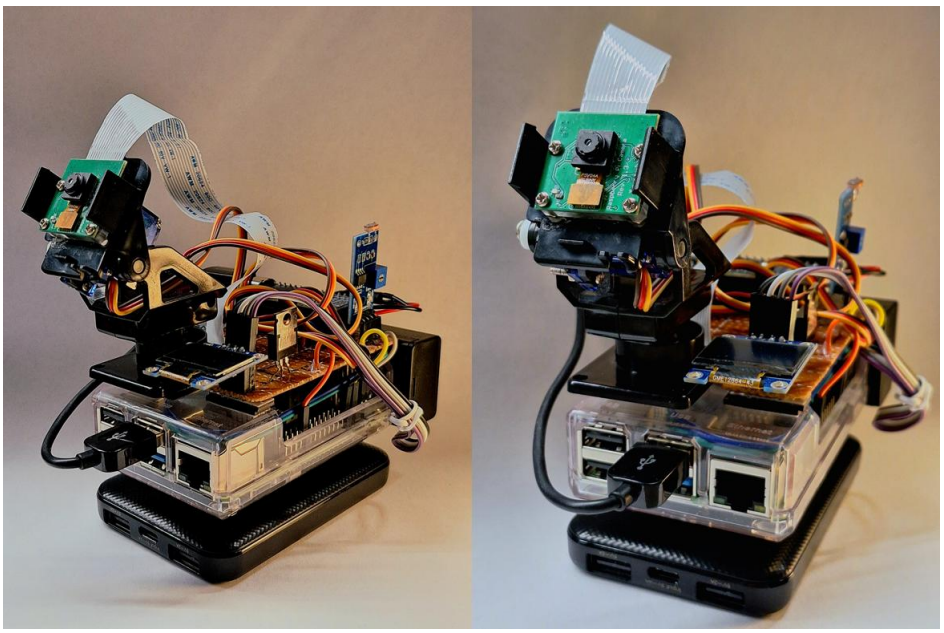


Figure 27: The design of the prototype translator device

Chapter 4: Results and Discussions

1. Overview

The result of this study is investigated further in this section, and an analysis of the accuracy of sign language translation achieved by the device prototype will be presented, offering valuable insights into the efficiency of the integration of sensors in the process of vision-based sign language translation, reaffirming the groundwork for future study.

2. Performance Analysis

In the experimental setup, once the hand assumes the appropriate position, the program promptly detects the corresponding sign language gesture. Subsequently, following a swift processing phase, the integrated system processes the quality of the ambient illumination surrounding the user and activates the lighting unit accordingly, the speaker component can be optionally triggered and emits a clear auditory output, effectively announcing the name of the interpreted letter as well as displaying the detected letter on the window monitor.

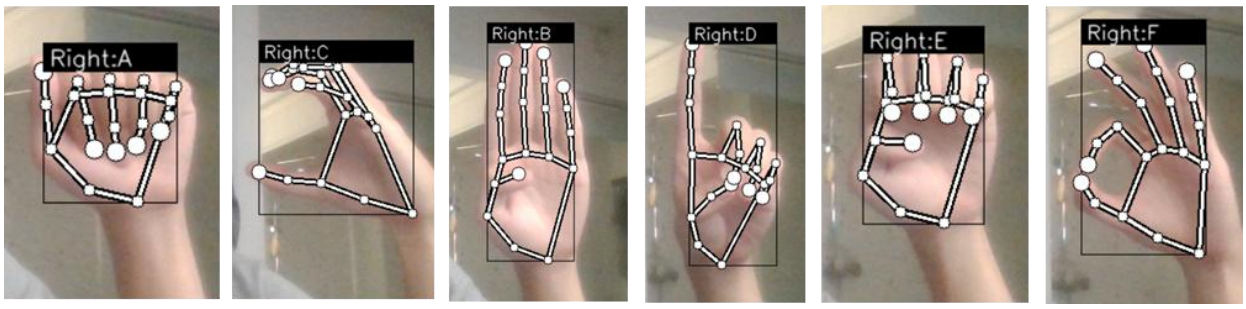


Figure 28: Recognition of the first 6 letters in the ASL alphabet

Concerning the accuracy of the hand recognition model, the program can recognize accurately a total of 24 letters in the English alphabet in ASL, excluding the two letters J and Z which are not static and require moving gestures to express. The current results are based on 30 tests per letter, with two male subjects and two female subjects, varying in the differences in hand sizes and lighting conditions. As expected the Mediapipe-based model works effectively for all hand types and even for lower lighting conditions. The only circumstance where the model failed to detect the hands of the user is when the surrounding environment either has extremely low illumination levels or practically none at all, in which case the lighting module will be of service.

The overall accuracy reaches approximately 90% for all test subjects, demonstrating excellent recognition capability with little to no errors or lags in between detections (Figure 28). The current mistakes include cases when the program fails to detect the hand, which are rare occurrences, or misunderstand one letter for another. One specific difficulty was the detection of the letters U and V, both share a similar hand posture leading to the failure in categorizing either of them. This was then compromised by slightly changing the position of the thumb on the letter U so that instead of facing inward into the palm, the thumb faces outward, as portrayed in Figure 29. This difference allows the landmark detection model to categorize the two hand signs as two distinguished letters.

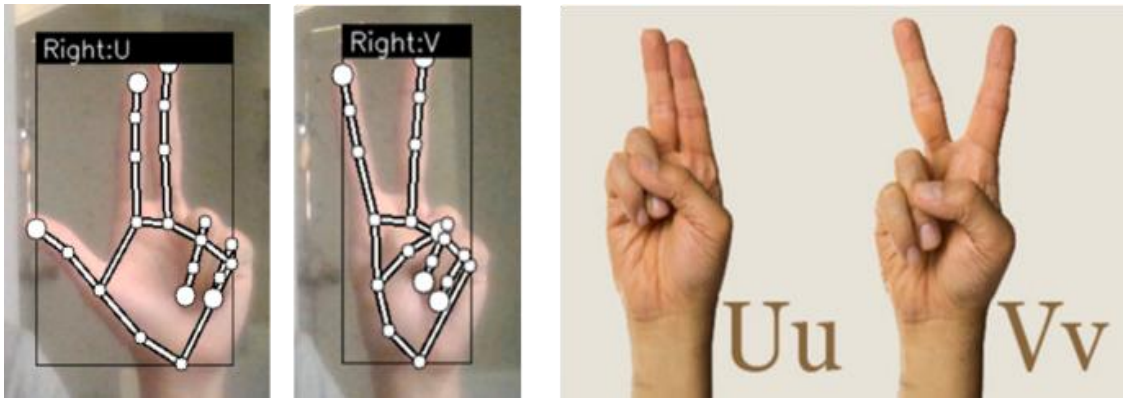


Figure 29: (Left): The newly proposed signing option (Right): The original posture

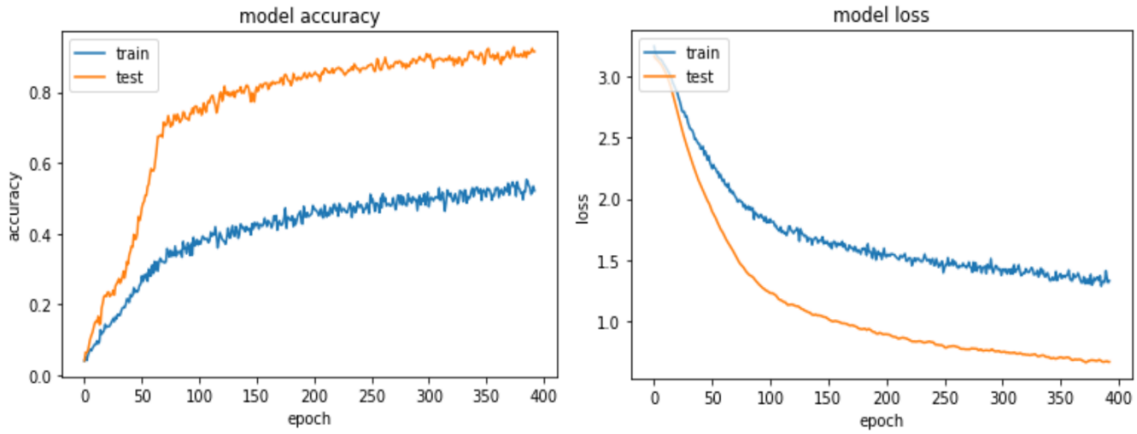
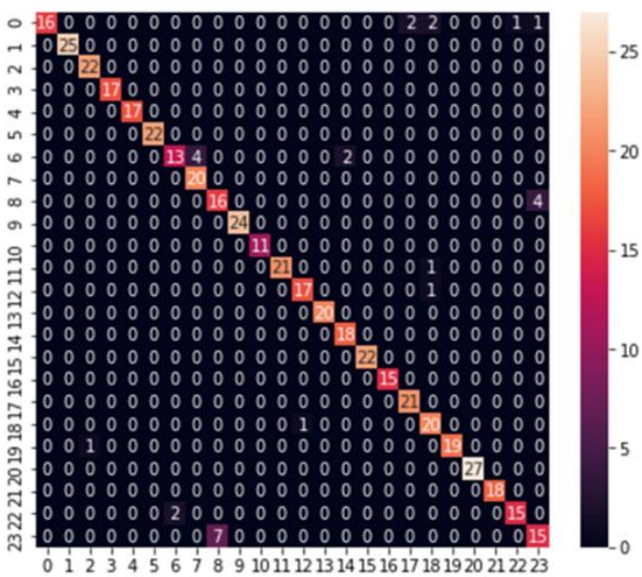


Figure 30: (Left): The accuracy graph after training and testing (Right): The loss of the model

The accuracy and the loss of the model are evidenced by the graphs generated using matplotlib, seen in Figure 30. It can be observed that on the loss over epochs plot, the test loss is significantly lower than the training loss, which is an uncommon occurrence. Given the nature of the model being trained on the training data, whereas the testing data is hidden, a more common and intuitive situation would involve the training loss being lower than the test loss. Similarly, the same can be seen in the model accuracy diagram where the accuracy of the validation is remarkably better than that of the training set. The reason for this is due to the dropout layers. As mentioned previously, the model includes two separate dropout layers, a 20% dropout after the input layer and a 40% dropout after the third layer. During the training of the model, the dropout layers set a specific amount of features to zero, this creates slight fluctuations in the line graph. Meanwhile, in the validation process, all the neurons are used. This leads to higher testing accuracies due to the model used during testing being more robust. Regardless of its atypicality, the results of the training and testing are still acceptable and well sufficient enough for hand-recognition applications. By using Sparse Categorial Cross Entropy, the model's loss was continuously reduced from about 3.5 to under 0.5. The confusion matrix in Figure 31 demonstrates the accuracy of the model on the test data for 24 different hand postures. Overall, the accuracy reached 94%, with the lowest f1-score for individual letters being 0.71. This score belongs to the letter Y due to its similarity in posture

with the letter I, which has an f1-score of 0.74. Both of the hand postures for these two letters share the extension of the little finger with the only difference being the position of the thumb, which for unknown reasons, Mediapipe has difficulty distinguishing. This case is similar to the one previously discussed with the letters U and V, although less severe and does not require any compromises. Surprisingly, the Mediapipe foundation works effectively even when occlusions are present in the posture, exemplified by the letters M and N, which the system has no issue distinguishing. Other than these cases, the rest of the signs from the ASL alphabet were detected and recognized with impressive accuracy, most having an f1-score of above 0.9.



Classification Report

	precision	recall	f1-score	support
0	1.00	0.73	0.84	22
1	1.00	1.00	1.00	25
2	0.96	1.00	0.98	22
3	1.00	1.00	1.00	17
4	1.00	1.00	1.00	17
5	1.00	1.00	1.00	22
6	0.87	0.68	0.76	19
7	0.83	1.00	0.91	20
8	0.70	0.80	0.74	20
9	1.00	1.00	1.00	24
10	1.00	1.00	1.00	11
11	1.00	0.95	0.98	22
12	0.94	0.94	0.94	18
13	1.00	1.00	1.00	20
14	0.90	1.00	0.95	18
15	1.00	1.00	1.00	22
16	1.00	1.00	1.00	15
17	0.91	1.00	0.95	21
18	0.83	0.95	0.89	21
19	1.00	0.95	0.97	20
20	1.00	1.00	1.00	27
21	1.00	1.00	1.00	18
22	0.94	0.88	0.91	17
23	0.75	0.68	0.71	22
accuracy			0.94	480
macro avg	0.94	0.94	0.94	480
weighted avg	0.94	0.94	0.94	480

Figure 31: The confusion matrix and the classification report of the model after 400 epochs

Concerning the effectiveness of the sensor and hardware implementation's assistance for the hand recognition model. The lighting unit employing the photosensor works as anticipated, although the Mediapipe library is robust enough to detect hand presence even in darker conditions, in cases where little to no light is present in the room, the unit provides enough illumination so that the system can detect and translate effectively. The servo-based pan and tilt unit also accomplishes its task sufficiently with the help of the Arduino microcontroller, positioning the camera so that it always captures the image of the user's hand once the model has detected its presence. The Arduino Nano controls the pan and tilt servos work together effectively with the program running on the Raspberry Pi to locate the positions of the detected hands and use the coordinates to adjust the servos-attached camera so that the user's hands are within the frame, allowing for non-disruptive image classification without the concern for limited space of operation that plagues stationary. Finally, the total cost of fabrication is approximately $90, substantially less than most popular sign translator products on the market, fulfilling the project's requirement for affordability.

3. Limitations

However, several limitations were encountered during the development and testing phases of the project. Especially the exclusion of two of the letters from the alphabet, J and Z, due to their gestures demanding the finger to move across the screen, tracking the history of the detected

landmark point of the pointer finger's tip, and training another model to perform specifically such task. This proves to be complicated and time-consuming and requires a lot more effort than the hand posture recognition model to be efficient and work at an acceptable level. Although this is not impossible to accomplish using the framework built by Mediapipe Hands, due to the time constraints of the project, this function was unfortunately elected to be dropped from the final prototype. Similarly, the issue with the inability to detect the difference between the thumb positions of the letters U and V remained something the Mediapipe library could improve upon. Although the current design offers a promising assortment of readable hand postures, its universal usability can be further improved by enriching the amount of data and consequently the number of recognizable signs. This introduces another limitation in the inability to translate more comprehensive words, which in many cases require precise motion of the hand to express correctly. Future iterations will aim to overcome this limitation by incorporating higher-level translation capabilities. Furthermore, expanding language support beyond English to include other languages, such as Vietnamese, is a crucial area for future development.

The framerates demonstrated when running the hand detection model on the Raspberry Pi 4 are also shown to be vastly lower than if the system was being run on higher-performance hardware. Although this issue has a relatively simple rectification, albeit not always available. By investing in a Google Coral USB Accelerator, the framerates can be significantly improved from merely averaging about 4 FPS to easily 35 FPS (on Tensorflow Lite, Raspberry Pi 4, 4GB RAM model)[50]. The drawback of this is the high cost of a USB Accelerator, with retail price charging around 60 USD for one module, that is if one can find one for a retail price. Nowadays, these items have been out of stock for some time, and the prices on the secondary market can be less than desirable. Given one of the objectives of this project is to create a cost-saving alternative to the usual highly expensive assistive types of technology, the decision to invest in this accelerator was not justified within the scope of this project. But perhaps it is a decision worth considering for anyone who wishes to improve the model for their own purposes in the future.

Finally, even though this project and many others that came before offered the user the ability to translate sign languages into other verbal languages, it is rarely the case that a device is proposed that allows the deaf to understand speaking languages. This phenomenon facilitates one-way communication, which limits the exclusivity that this field of technology is meant to provide. There were ideas about how this project could attempt to tackle this issue, but regrettably none of which came to fruition due to the disproportional ratio between the number of research done on sign-to-speech translation compared to the other way around. Considering the amount of time that was given to this project, it is also difficult to maintain efficiency and precision across the system with the amount of combined functionalities that it would have been attempting to juggle.

The prevalence of deafness in today's society underscores the importance of such assistive technologies. Despite the widespread need, there remains a lack of adequate support and the spreading of awareness to make technology and communication more accessible for the deaf community, especially in resource-constrained settings. By focusing on affordability and accessibility, this project seeks to help create a meaningful impact on the lives of individuals with hearing or speech disabilities.

Chapter 5: Conclusion

With the rapid development in motion capture and sensor technology, the future of language interpreter devices will no doubt continue to grow and advance to new areas too vast to cover in one project. Regardless, the sign language translator prototype demonstrated promising results in converting hand signs into audible or displayable words, coupled with the integration of a photosensor and hand tracking pan and tilt camera, this combination managed to address and rectify several of the issues commonly mentioned in conversations regarding image-based hand recognition. The results of this project are further proof that a more augmented design of this device based on these fundamental principles would undoubtedly be able to accurately and effectively interpret a wider range of gestures from any sign language system. The potential of this method in hand sign translation could be realized once the points previously addressed are remedied, such as the problem with lower framerates, implementing motion gesture recognition, making the device more portable, and allowing for two-way communication.

However, the relative success in the development and testing of the device indicates a positive step towards inclusivity and equal access to information, facilitating better understanding between people of different abilities by integrating this system into educational purposes, increasing independence for disabled people in social circumstances, enabling the use of home products or previously-voice-based technology for vocally-ineffective people and hearing-impaired individuals. Moving forward, improvements and enhancements can be made with further research and development, allowing for a more user-friendly design with improved accuracy as well as a larger, more complex selection of gestures available for translation.

All things considered, due to hardware and budget limitations, there are many more advanced technologies that can be adopted into this design, creating an optimal device with increased accuracy, portability, and affordability. The involvement of more state-of-the-art technologies in assistive technologies is all under research [51]. Despite these challenges, the project accomplished its objectives by addressing and offering a viable solution for several criticisms made against the hand recognition method of employing vision-based technology and machine learning, mainly the issue with varying lighting conditions and self-occlusion affecting the accuracy of detection as well as the restricting space in which the hand of the user needs to be in for the program to recognize its positions and gestures. The offered answers to these problems were: (1) to employ a sensor that detects the ambient lighting and activates additional built-in lights accordingly, (2) exploit the existing robust Mediapipe hand detection library when building the recognition model, and (3) install a pan-and-tilt unit for the camera unit so that once a hand is detected within the detecting frame, the unit will automatically rotate across its axes in order to accommodate the new position of the user's hand. The development of this project's sign language translator offers an idea that may be useful for further future research and fleshing out in the field of assistive technology. With additional recommended refinement and innovation, there is enough potential in vision-based hand recognition technology to significantly enhance communication and exclusivity for many individuals worldwide.

References

[1] "Deafness and hearing loss." Accessed: Sep. 11, 2024. [Online]. Available: https://www.who.int/health-topics/hearing-loss

[2] J. Hill, "Do deaf communities actually want sign language gloves?," *Nat Electron*, vol. 3, no. 9, pp. 512–513, Jul. 2020, doi: 10.1038/s41928-020-0451-7.

[3] "Light invariant real-time robust hand gesture recognition," *Optik*, vol. 159, pp. 283–294, Apr. 2018, doi: 10.1016/j.ijleo.2017.11.158.

[4] V. A. Shanthakumar, C. Peng, J. Hansberger, L. Cao, S. Meacham, and V. Blakely, "Design and evaluation of a hand gesture recognition approach for real-time interactions," *Multimed Tools Appl*, vol. 79, no. 25, pp. 17707–17730, Jul. 2020, doi: 10.1007/s11042-019-08520-1.

[5] "Systematic Literature Review: American Sign Language Translator," *Procedia Computer Science*, vol. 179, pp. 541–549, Jan. 2021, doi: 10.1016/j.procs.2021.01.038.

[6] S. Parihar, N. Shrotriya, and P. Thakore, "Hand Gesture Recognition: A Review," in *Proceedings of 3rd International Conference on Mathematical Modeling and Computational Science*, S.-L. Peng, N. Z. Jhanjhi, S. Pal, and F. Amsaad, Eds., Singapore: Springer Nature, 2023, pp. 471–483. doi: 10.1007/978-981-99-3611-3_39.

[7] A. Kurakin, Z. Zhang, and Z. Liu, "A real time system for dynamic hand gesture recognition with a depth sensor," in *2012 Proceedings of the 20th European Signal Processing Conference (EUSIPCO)*, Aug. 2012, pp. 1975–1979. Accessed: Sep. 12, 2024. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/6333871

[8] "A Review of the Hand Gesture Recognition System: Current Progress and Future Directions | IEEE Journals & Magazine | IEEE Xplore." Accessed: Sep. 13, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/9622242

[9] K. S. Abhishek, L. C. F. Qubeley, and D. Ho, "Glove-based hand gesture recognition sign language translator using capacitive touch sensor," in *2016 IEEE International Conference on Electron Devices and Solid-State Circuits (EDSSC)*, Aug. 2016, pp. 334–337. doi: 10.1109/EDSSC.2016.7785276.

[10] E. Jeong, J. Lee, and D. Kim, "Finger-gesture Recognition Glove using Velostat (ICCAS 2011)," in *2011 11th International Conference on Control, Automation and Systems*, Oct. 2011, pp. 206–210. Accessed: Sep. 13, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/6106275

[11] N. Praveen, N. Karanth, and M. S. Megha, "Sign language interpreter using a smart glove," in *2014 International Conference on Advances in Electronics Computers and Communications*, Oct. 2014, pp. 1–5. doi: 10.1109/ICAECC.2014.7002401.

[12] O. Nisar, M. A. Imtiaz, S. Hussain, and O. Saleem, "Performance Optimization of a Flex Sensor Based Glove for Hand Gestures Recognition and Translation," *International Journal of Engineering Research*, vol. 3, no. 5, 2014.

[13] Y. Bai, Z. Zhang, H. Wang, R. Guo, and X. Li, "Sawtooth-enhanced bend sensor for gesture recognition," *Sci. China Technol. Sci.*, vol. 67, no. 6, pp. 1727–1736, Jun. 2024, doi: 10.1007/s11431-023-2621-5.

[14] A. K. Panda, R. Chakravarty, and S. Moulik, "Hand Gesture Recognition using Flex Sensor and Machine Learning Algorithms," in *2020 IEEE-EMBS Conference on Biomedical Engineering and Sciences (IECBES)*, Mar. 2021, pp. 449–453. doi: 10.1109/IECBES48179.2021.9398789.

[15] A. I. M. Thaim, N. Sazali, K. Kadirgama, A. S. Jamaludin, F. M. Turan, and N. A. Razak, "Smart Glove for Sign Language Translation," *Journal of Advanced Research in Applied Mechanics*, vol. 112, no. 1, Art. no. 1, Jan. 2024, doi: 10.37934/aram.112.1.8087.
[16] "Sign language translator and gesture recognition | IEEE Conference Publication | IEEE Xplore." Accessed: Sep. 13, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/7353332
[17] P. Trigueiros, F. Ribeiro, and L. Reis, "Hand Gesture Recognition System Based in Computer Vision and Machine Learning," in *Lecture Notes in Computational Vision and Biomechanics*, vol. 19, 2013, pp. 137–142. doi: 10.1007/978-3-319-13407-9_21.
[18] D. Guo, W. Zhou, M. Wang, and H. Li, "Sign language recognition based on adaptive HMMS with data augmentation," in *2016 IEEE International Conference on Image Processing (ICIP)*, Phoenix, AZ, USA: IEEE, Sep. 2016, pp. 2876–2880. doi: 10.1109/ICIP.2016.7532885.
[19] W. Aly, S. Aly, and S. Almotairi, "User-Independent American Sign Language Alphabet Recognition Based on Depth Image and PCANet Features," *IEEE Access*, vol. 7, pp. 123138–123150, 2019, doi: 10.1109/ACCESS.2019.2938829.
[20] Z. Ren, J. Yuan, J. Meng, and Z. Zhang, "Robust Part-Based Hand Gesture Recognition Using Kinect Sensor," *IEEE Transactions on Multimedia*, vol. 15, no. 5, pp. 1110–1120, Aug. 2013, doi: 10.1109/TMM.2013.2246148.
[21] S. D. Badgujar, G. Talukdar, O. Gondhalekar, and M. S. Y. Kulkarni, "Hand Gesture Recognition System," vol. 4, no. 2, 2014.
[22] F. Zhang *et al.*, "MediaPipe Hands: On-device Real-time Hand Tracking," Jun. 17, 2020, *arXiv*: arXiv:2006.10214. Accessed: Sep. 14, 2024. [Online]. Available: http://arxiv.org/abs/2006.10214
[23] A. R. Verma, G. Singh, K. Meghwal, B. Ramji, and P. K. Dadheech, "Enhancing Sign Language Detection through Mediapipe and Convolutional Neural Networks (CNN)," Aug. 27, 2024, *arXiv*: arXiv:2406.03729. doi: 10.48550/arXiv.2406.03729.
[24] "American Sign Language Recognition for Alphabets Using MediaPipe and LSTM," *Procedia Computer Science*, vol. 215, pp. 642–651, Jan. 2022, doi: 10.1016/j.procs.2022.12.066.
[25] M. E. Morocho-Cayamcela and W. Lim, "Fine-tuning a pre-trained Convolutional Neural Network Model to translate American Sign Language in Real-time," Jan. 2019, pp. 100–104. doi: 10.1109/ICCNC.2019.8685536.
[26] "What Is American Sign Language (ASL)? | NIDCD." Accessed: Sep. 12, 2024. [Online]. Available: https://www.nidcd.nih.gov/health/american-sign-language
[27] "raspberry-pi-4-datasheet.pdf." Accessed: Sep. 18, 2024. [Online]. Available: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf
[28] "Raspberry Pi Camera Rev 1.3 Philippines," circuitrocks. Accessed: Sep. 18, 2024. [Online]. Available: https://circuit.rocks/products/raspberry-pi-camera
[29] Matt, "Raspberry Pi Camera Module Mechanical Dimensions," Raspberry Pi Spy. Accessed: Sep. 18, 2024. [Online]. Available: https://www.raspberrypi-spy.co.uk/2013/05/pi-camera-module-mechanical-dimensions/
[30] "Nano | Arduino Documentation." Accessed: Sep. 18, 2024. [Online]. Available: https://docs.arduino.cc/hardware/nano/#tech-specs
[31] "Introduction to Debian Linux," GeeksforGeeks. Accessed: Sep. 18, 2024. [Online]. Available: https://www.geeksforgeeks.org/introduction-to-debian-linux/

[32] H.-C. Steiner, “Firmata: Towards Making Microcontrollers Act Like Extensions of the Computer”.
[33] “LDR Module - LM393,” ADIY. Accessed: Sep. 19, 2024. [Online]. Available: https://adiy.in/shop/ldr-modulelight-dependent-resistorlm393/
[34] “Servo Motor (SG90) — SunFounder Ultimate Sensor Kit documentation.” Accessed: Sep. 19, 2024. [Online]. Available: https://docs.sunfounder.com/projects/ultimate-sensor-kit/en/latest/components_basic/27-component_servo.html
[35] “Servo Motor S MG90S (Metal Gears, Digital),” Smart Prototyping. Accessed: Sep. 19, 2024. [Online]. Available: https://www.smart-prototyping.com/MG90S-9g-metal-gear-digital-servo.html
[36] “GitHub - kinivi/hand-gesture-recognition-mediapipe: This is a sample program that recognizes hand signs and finger gestures with a simple MLP using the detected key points. Handpose is estimated using MediaPipe.” Accessed: Sep. 17, 2024. [Online]. Available: https://github.com/kinivi/hand-gesture-recognition-mediapipe?tab=readme-ov-file
[37] F. Zhang *et al.*, “MediaPipe Hands: On-device Real-time Hand Tracking,” Jun. 17, 2020, *arXiv*: arXiv:2006.10214. Accessed: Sep. 14, 2024. [Online]. Available: http://arxiv.org/abs/2006.10214
[38] “Hierarchical Data Format - Wikipedia.” Accessed: Sep. 20, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Hierarchical_Data_Format
[39] “Multilayer Perceptrons in Machine Learning: A Comprehensive Guide.” Accessed: Sep. 20, 2024. [Online]. Available: https://www.datacamp.com/tutorial/multilayer-perceptrons-in-machine-learning
[40] “Introduction to Activation Functions in Neural Networks.” Accessed: Sep. 20, 2024. [Online]. Available: https://www.datacamp.com/tutorial/introduction-to-activation-functions-in-neural-networks
[41] P. Belagatti, “Understanding the Softmax Activation Function: A Comprehensive Guide,” SingleStore. Accessed: Oct. 01, 2024. [Online]. Available: https://www.singlestore.com/blog/a-guide-to-softmax-activation-function/
[42] “What Is Gradient Descent?,” Built In. Accessed: Sep. 20, 2024. [Online]. Available: https://builtin.com/data-science/gradient-descent
[43] D. A. Das, “This blog post aims at explaining the behavior of different algorithms for optimizing gradient…,” Analytics Vidhya. Accessed: Oct. 01, 2024. [Online]. Available: https://medium.com/analytics-vidhya/this-blog-post-aims-at-explaining-the-behavior-of-different-algorithms-for-optimizing-gradient-46159a97a8c1
[44] “Loss Functions in Machine Learning Explained.” Accessed: Sep. 21, 2024. [Online]. Available: https://www.datacamp.com/tutorial/loss-function-in-machine-learning
[45] S. Kedia and P. Nath, “Here is what you need to know about Sparse Categorical Cross Entropy in nutshell,” Machine Learning Diaries. Accessed: Sep. 21, 2024. [Online]. Available: https://vevesta.substack.com/p/here-is-what-you-need-to-know-about
[46] “Cross-Entropy Loss Function in Machine Learning: Enhancing Model Accuracy.” Accessed: Sep. 21, 2024. [Online]. Available: https://www.datacamp.com/tutorial/the-cross-entropy-loss-function-in-machine-learning
[47] M. Sorokin, “Categorical Cross-Entropy: Unraveling its Potentials in Multi-Class Classification,” Medium. Accessed: Sep. 21, 2024. [Online]. Available: https://medium.com/@vergotten/categorical-cross-entropy-unraveling-its-potentials-in-multi-class-classification-705129594a01

[48] H. Yadav, “Dropout in Neural Networks,” Medium. Accessed: Sep. 21, 2024. [Online]. Available: https://towardsdatascience.com/dropout-in-neural-networks-47a162d621d9

[49] “The Concepts of Dense and Sparse in the Context of Neural Networks | Baeldung on Computer Science.” Accessed: Sep. 21, 2024. [Online]. Available: https://www.baeldung.com/cs/neural-networks-dense-sparse/

[50] “TensorFlow-Lite-Object-Detection-on-Android-and-Raspberry-Pi/deploy_guides/Raspberry_Pi_Guide.md at master · EdjeElectronics/TensorFlow-Lite-Object-Detection-on-Android-and-Raspberry-Pi,” GitHub. Accessed: Sep. 25, 2024. [Online]. Available: https://github.com/EdjeElectronics/TensorFlow-Lite-Object-Detection-on-Android-and-Raspberry-Pi/blob/master/deploy_guides/Raspberry_Pi_Guide.md

[51] M. Papatsimouli *et al.*, “Real Time Sign Language Translation Systems: A review study,” Jun. 2022, pp. 1–4. doi: 10.1109/MOCAST54814.2022.9837666.